

基于情景树与帕累托前沿的多目标供应链优化方案

摘要

在实际生产中，采购单价、运输费用、运输损耗及存储费用都直接影响企业的生产效益。同时，企业要尽可能维持库存水平以保证生产连续性。为此本文基于近 5 年 402 家供应商以及 8 家转运商的数据，综合考虑供应能力、采购成本、运输损耗、库存安全和供应链风险，构建由供应商综合评价-随机需求订购优化-动态损耗转运优化-产能成本风险决策组成的一体化模型，为企业未来 24 周的原材料采购与运输提供决策依据。

针对问题一，本问首先基于 99.5%分位数 Winsorize 截尾法处理极端数据。其次，从供货能力、平均供货强度、供货波动、到货率、订单满足率和供货覆盖率六个维度构造供应商评价指标体系。接着，针对部分指标之间存在较强相关性的特点，采用 CRITIC 与熵权法进行组合赋权。最后，结合 TOPSIS 模型计算 402 家供应商的综合贴近度，从而筛选出前 50 家重要供应商。

针对问题二，基于问题一本问首先以历史同期数据刻画未来 24 周需求波动，并利用拉丁超立方抽样生成 500 条需求情景树。其次，引入 Mean-CVaR 风险度量，在期望采购成本与极端情景风险之间进行权衡，确定最少供应商下的 24 周订购方案；接着，建立整数规划转运模型，在满足转运商运力的前提下最小化运输损耗。最后，输出了仅付出较小的期望成本代价即可有效压缩尾部风险的转运方案，从而实现成本、库存安全和供应商选择之间的协调。

针对问题三，本问基于问题二模型重新构造材料采购结构，并采用字典序优化，优化后 A 类原材料采购占比由 31.7%提高至 100%，首周因初始库存为空出现约 5711m 等价产品的安全库存缺口，但第 2 周起即可恢复并持续满足库存要求。针对转运环节，采用历史同期加权预测并引入样本量自适应收缩机制，再构建逐周 MILP 进行运输分配。最终方案优先使用低损耗的 T3、T6，整体加权平均损耗率约为 0.34%，24 周预计总损耗约为 1535.8m³。

针对问题四，本问首先从转运商和供应商能力角度确定产能理论边界，其中转运端可支持的理论产品产能约为 6.58 万~7.89 万 m³/周。其次，以企业产能为扫描变量，建立综合考虑总成本、供应链集中度 HH1 和安全库存的产能-成本-风险 Pareto 分析模型，并利用边际成本弹性和风险变化率识别扩产拐点。最终推荐企业将周产能提高至 4.60 万 m³/周，提高约 63.12%，在明显提升产能的同时保持供应链风险和库存安全处于可控范围。

本文的亮点在于：首先，完整描述了“供应商评价-订购-转运-扩产”这一实际生产决策过程；其次，将需求随机性和尾部风险纳入订购模型，使订购方案不仅追求低成本，也能避免极端需求情景造成严重缺货；最后，产能推荐值具有明确的成本-风险权衡意义，更符合企业扩产中的实际决策逻辑。

关键词：CRITIC+熵权法组合赋权 情景树+Mean-CVaR 字典序优化 Pareto 三维分析模型+扩产拐点

一、 问题重述

1.1 问题背景

木质纤维和其他植物纤维材料作为某企业在建造及装修板材的生产中所用的主要原材料，将其分为 A，B，C 三类。该企业计划 48 周为生产周期，为此需制定为期 24 周的原材料订购及转运计划，即按照产能要求确定每周所需的原材料订货量和相应的供货商，委托确定的转运商将供应商每周的供货量转运至企业仓库。

该企业每立方米产品需要消耗原材料 0.6 立方米 A 类、0.66 立方米 B 类、或 0.72 立方米 C 类，每周产能为 28200 立方米。因原材料具有特殊性，故供应商无法严格保证按照订货量供货，实际供货量与订货量存在随机偏差。该企业要维持至少满足两周生产需求的原材料库存量以保证生产连续性，为此该企业总是全部收购供应商实际提供的原材料，以期尽可能维持库存水平。

1.2 问题提出

在转运的实际过程中存在损耗，定义损耗率为损耗量占供货量的比值，转运商实际送到仓库的原材料数量为接收量。每家转运商的运输能力均为 6000 立方米/周。一家供应商每周供应的原材料由一家转运商运输。企业的生产效益受到原材料采购成本的直接影响，原材料 A 类采购单价为 C 类的 1.2 倍，B 类为 C 类的 1.1 倍，而运输及存储的单位费用相同。

基于附件 1 中该企业 402 家供应商近 5 年的订货量和供货量数据，以及附件 2 中 8 家转运商的运输损耗率数据，进行深入分析并解决以下问题：

问题一：根据附件 1，建立量化评估供应商对保障企业生产重要性的数学模型，该模型应综合考虑供应商的供货稳定性、供货能力及供货可靠性等指标。根据模型评分排序，并在论文中列表给出重要性排名前 50 的供应商。

问题二：基于问题 1，确定满足生产需求所需的最少供应商数量。据此，为企业制定未来 24 周，以总采购成本最低为目标的订购方案和损耗率最小为目标的转运方案，并分析两方案的实施效果。

问题三：为压缩生产成本，企业拟调整原材料采购结构：多采购 A 类原材料和少采购 C 类原材料。以总成本最低和转运损耗率最少为目标，制定新的订购及转运方案，并对方案的实施效果进行分析。

问题四：企业通过技术革新具备了提高产能的潜力。在目前供应商和转运商的实际情况约束下，确定该企业每周产能增长范围，并制定未来 24 周最优的订购和转运方案。

二、 问题分析

2.1 问题一分析

问题要求我们基于附件 1，建立量化评估供应商对保障企业生产重要性的数学模型，并根据模型评分排序给出重要性排名前 50 的供应商。本文首先对数据中进行清洗，识别并剔除主动补货记录，并采用 Winsorize 截尾法处理极端值。其次，构建六个维度指标评价体系并进行 Pearson 相关性检验。接着，采用 CRITIC+熵权法+最小二乘法组合赋权。最后，输出供应商综合排名，从而得到重要性排名前 50 的供应商。

2.2 问题二分析

问题要求我们基于问题 1，确定满足生产需求的最少供应商数量，并为该企业制定未来 24 周，总采购成本最低的订购方案和损耗率最小的转运方案，且分析两方案的实施效果。这首先要求我们从题干中提取基础数据并定义参数。其次，采用拉丁超立方抽样构建三叉情景树。接着，结合 Mean-CVaR 输出总采购成本最低的订购方案。最后，运用线性整数规划和贪心算法，从而得出损耗率最小的转运方案。

2.3 问题三分析

问题要求我们多采购 A 类原材料和少采购 C 类原材料，以压缩生产成本。以总成本最低和转运损耗率最少为目标，制定新的订购及转运方案，并对方案的实施效果进行分析。这首先要求我们在建立模型前说明决策变量和约束条件。其次，构建目标函数输出新的订购方案。接着，最小化 24 周预测总损耗。最后，用逐周动态损耗率进行 MILP 分配，从而得出新的转运方案。

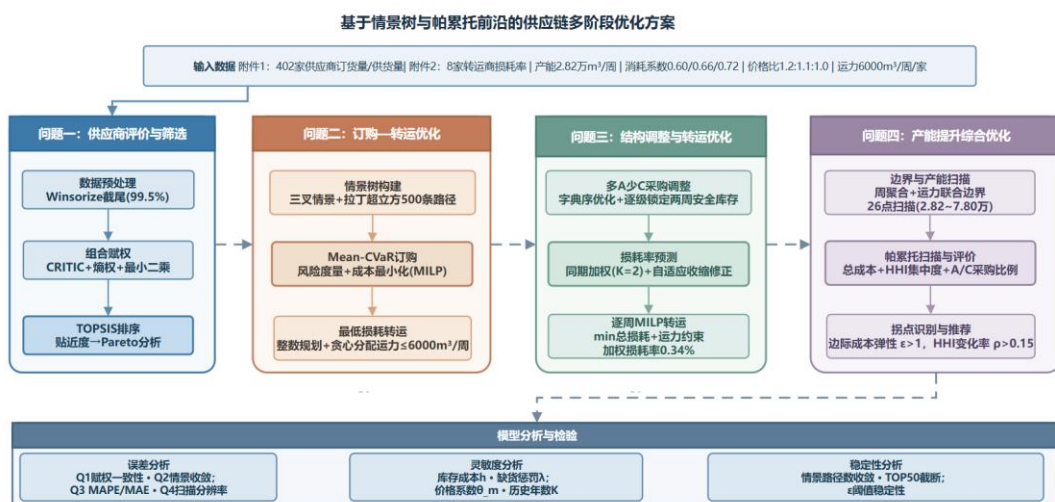
2.4 问题四分析

问题要求我们在目前供应商和转运商的实际情况约束下，确定该企业每周产能增长范围，并制定未来 24 周最优的订购和转运方案。这首先要求我们从确定转运商运力和供应商端理论边界约束。其次，构建产能-成本-风险帕累托分析模型。接着，进行拐点识别与产能推荐。最后，输出未来 24 周最优的订购和转运方案。

2.5 总体思路

本篇文章总体思路如图 1。

图 1 总体思路图



三、 模型假设

- 1、假设供应商实际给转运商的货物数量一定为非负数。
- 2、供应商的历史供货行为具有各自代表性。过去 240 周的历史数据能够反映未来 24 周内各供应商的供货情况，不存在结构性突变。
- 3、转运商的损耗率可通过历史同期数据进行有效预测。损耗率在同一历史同期窗口内的损耗率具有统计学意义。
- 4、供应商的供货偏差与订购量规模无关。

四、 符号说明

符号	说明
o_{ij}	第 i 家供应商第 j 周的订货量
s_{ij}	第 i 家供应商第 j 周的供货量
Q_{it}	总供货量
l_k	各转运商具体损耗率
I_{tm}	第 t 周末 m 类原材料库存
x_m	每天实际运输多少 m 类原材料
CT	在约束下的产能

五、 问题一的模型建立与求解

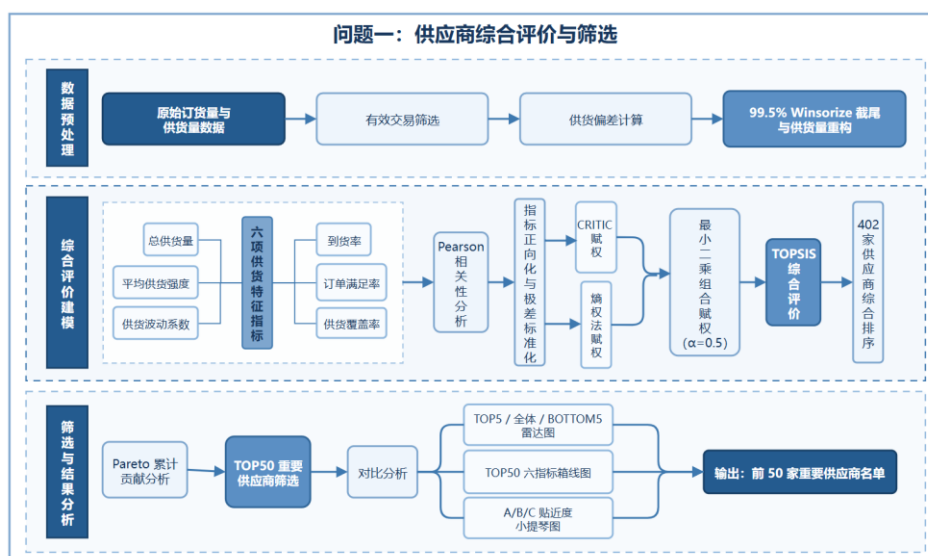


图 2 问题一求解思路图

5.1 数据预处理

在进行数据处理前，为确保数据的可用性、可靠性及科学性，我们首先对订货量为 0 但供货量不为 0 的情况进行排查，并定义为**主动补货**，不参与后续差值统计，交易记录类型分布如图 3 所示。

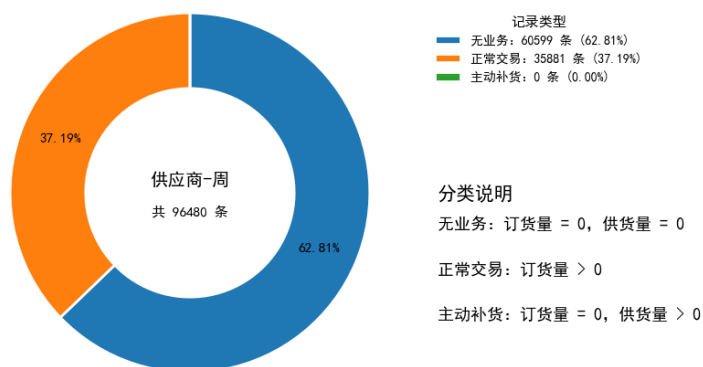


图 3 供应商交易记录类型分布图

据图 3 我们发现主动补货记录为 0 次，不存在这种异常供货情况。其次，对其余记录进行订货量与供货量之间的差值统计，并绘制差值分布直方图+核密度曲线，如图 4 所示。

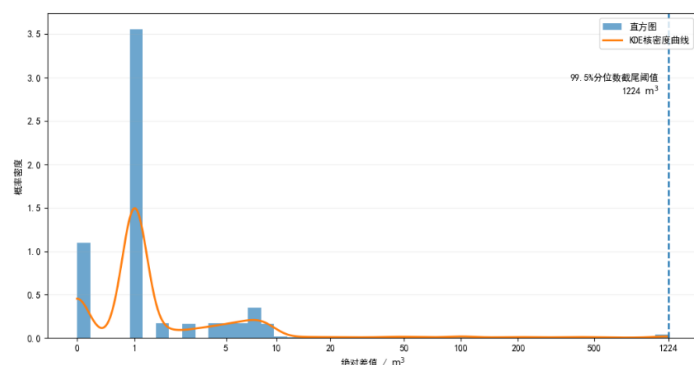


图 4 订货量与供货量绝对差值分布图

对图 4 进行分布形态与长尾特征分析可知：订货量与供货量绝对差值分布呈右偏长尾形态；大多数订单供需匹配精度较高，但存在少部分异常补货或数据记录不一致的情况。

接着，为消除极端值对后续建模的干扰，本文采用 Winsorize 截尾法，以 99.5%分位数作为截断阈值。为进一步量化截尾效果，本文统计了 Winsorize 处理的关键数据如表 1 所示。

表 1 Winsorize 截尾处理统计

指标	数值
99.5%分位数截尾阈值	1224.0000
被截尾记录数	180.0000
被截尾比例 (%)	0.5017
截尾后最大差值	1224.0000

由表 1 中数据可知，极端值属于稀有事件，截尾操作既有效消除极端干扰，又保留了 99.5%的样本原始信息，处理方式稳妥可信。

基于以上步骤，本文对差值做对数变换 $\log(x + 1)$ ，用于可视化对比三类材料，输出差值箱线图对比如图 5。

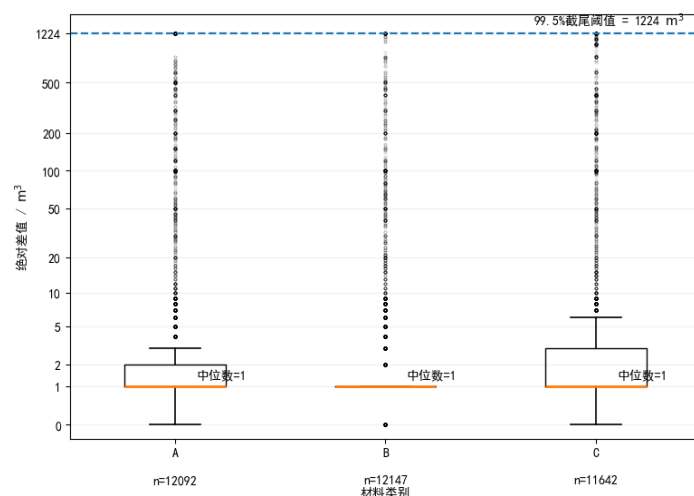


图 5 三类原材料差值箱线图

据图 5 可得：C 类分布最为集中，波动最小，供货匹配精度最高；A、B 两类形态相近，但 A 类偏差幅度相对较大；三类材料整体供货质量良好。

5.2 构建供应商评价指标体系

为从不同维度量化供应商表现，本文选取了 6 项指标。

总供货量（截尾后）：

$$W_i = \sum_{j=1}^{240} s_{ij} \quad (1)$$

平均供货强度（截尾后）：

$$\bar{s}_i = \frac{W_i}{T_i} \quad (2)$$

供货波动系数：

$$CV_i = \frac{\sigma(s_i)}{\bar{s}_i} \quad (3)$$

到货率：

$$R_i = \frac{W_i}{O_i^{\text{有效}}} \quad (4)$$

订单满足率：

$$SR_i = \frac{N_i^{\text{满足}}}{N_i^{\text{有效}}} \quad (5)$$

供货覆盖率：

$$c_i = \frac{T_i}{240} \quad (6)$$

其中， o_{ij} 、 s_{ij} 为第 i 家供应商第 j 周的订货量和供货量； T_i 为实际供货周数；

$\sigma(s_i)$ 为供货量标准差； $O_i^{\text{有效}}$ 为有订货需求的周次对应的订货量之和； $N_i^{\text{有效}}$ 为企业有订货的周次数； $N_i^{\text{满足}}$ 为供货量 $\geq$ 订货量 $\times 0.9$ 的周次数。

为避免 6 个指标间信息冗余影响赋权准确性，本文对 6 个指标进行 Pearson 相关性检验如图 6 所示。



图 6 六项供应商供货特征相关系数热力图

由图 6 可知，到货率与订单满足率、订单满足率与供货覆盖率、到货率与供货覆盖率之间存在较强的正相关关系。若直接采用仅依赖信息量差异的赋权方法，将导致相关指标被重复赋权，从而影响评价结果。因此，本文引入组合赋权，以有效避免信息冗余产生偏差。

5.3 组合赋权

5.3.1 数据标准化

为消除量纲，统一指标方向，本文进行数据标准化处理。设评价矩阵为 $X = (x_{ij})_{402 \times 6}$ ，考虑到 TOPSIS 逼近理想解排序法并满足它的基本假设，本文采用极差标准化法。

正向指标 (1, 2, 3, 4, 5, 6)：

$$y_{ij} = \frac{x_{ij} - \min_j(x_{ij})}{\max_j(x_{ij}) - \min_j(x_{ij})} \quad (7)$$

负向指标 (3)：

$$y_{ij} = \frac{\max_j(x_{ij}) - x_{ij}}{\max_j(x_{ij}) - \min_j(x_{ij})} \quad (8)$$

5.3.2 CRITIC 赋权法

$$H_j = \sigma_j \cdot \sum_{k=1}^6 (1 - \rho_{jk}) \quad (9)$$

$$w_j^{CRITIC} = \frac{H_j}{\sum_{k=1}^6 H_k} \quad (10)$$

其中， σ_j 为标准差， ρ_{jk} 为相关系数。

5.3.3 熵权法赋权

$$P_{ij} = \frac{y_{ij}}{\sum_{i=1}^{402} y_{ij}} \quad (11)$$

$$e_j = -\frac{1}{\ln 402} \sum_{i=1}^{402} P_{ij} \ln P_{ij} \quad (12)$$

$$w_j^{Entropy} = \frac{1 - e_j}{\sum_{k=1}^6 (1 - e_k)} \quad (13)$$

5.3.4 组合权重

$$\widehat{w}_j = \alpha \cdot w_j^{CRITIC} + (1 - \alpha) w_j^{Entropy} \quad (14)$$

用最小二乘法求出 α ：

$$\alpha = \max_{\alpha} \sum_{j=1}^6 \left[(\widehat{w}_j - w_j^{CRITIC})^2 + (\widehat{w}_j - w_j^{Entropy})^2 \right] \quad (15)$$

输出 6 个指标三种权重对比柱状图如图 7。

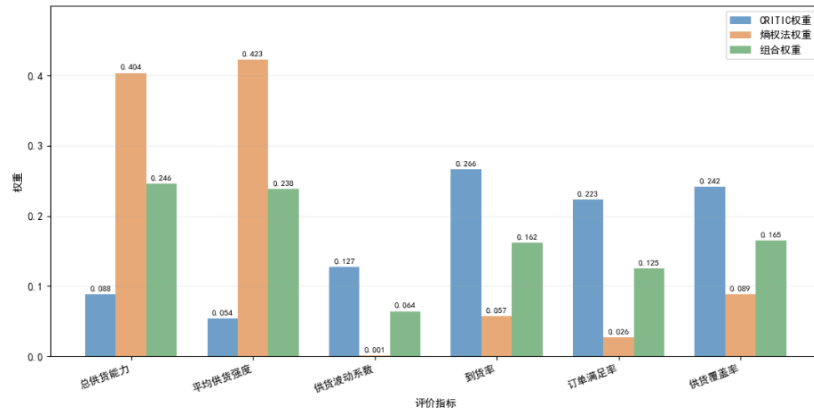


图 7 六项供应商评价指标客观权重对比图

由图 7 可知：三种赋权方法的结果存在显著差异，CRITIC 法^[1]侧重指标间的相关性结构，熵权法^[2]偏好数据离散程度，而组合权重融合了两者优先。单一方法存在系统性偏好，组合赋权^[3]结果更稳健合理，可作为后续 TOPSIS 综合评价的权重依据。

5.4 TOPSIS 综合评价与排序

为得出供应商的综合得分与排名，本文设加权决策矩阵为 $V = (v_{ij})_{402 \times 6}$ ，其中 $v_{ij} = \widehat{w}_j \cdot y_{ij}$ 。

正负理想解分别为：

$$V^+ = (v_1^+, v_2^+, \dots, v_6^+), v_j^+ = \max_i v_{ij} \quad (16)$$

$$V^- = (v_1^-, v_2^-, \dots, v_6^-), v_j^- = \min_i v_{ij} \quad (17)$$

各供应商到正负理想解的欧氏距离为：

$$D_i^+ = \sqrt{\sum_{j=1}^6 (v_{ij} - v_j^+)^2} \quad (18)$$

$$D_i^- = \sqrt{\sum_{j=1}^6 (v_{ij} - v_j^-)^2} \quad (19)$$

贴近度为：

$$S_i = \frac{D_i^-}{D_i^+ + D_i^-}, S_i \in [0, 1] \quad (20)$$

基于以上步骤输出 TOPSIS^[4]正负理想解散点图及贴合度分布直方图与核密度曲线，如图 8 所示。

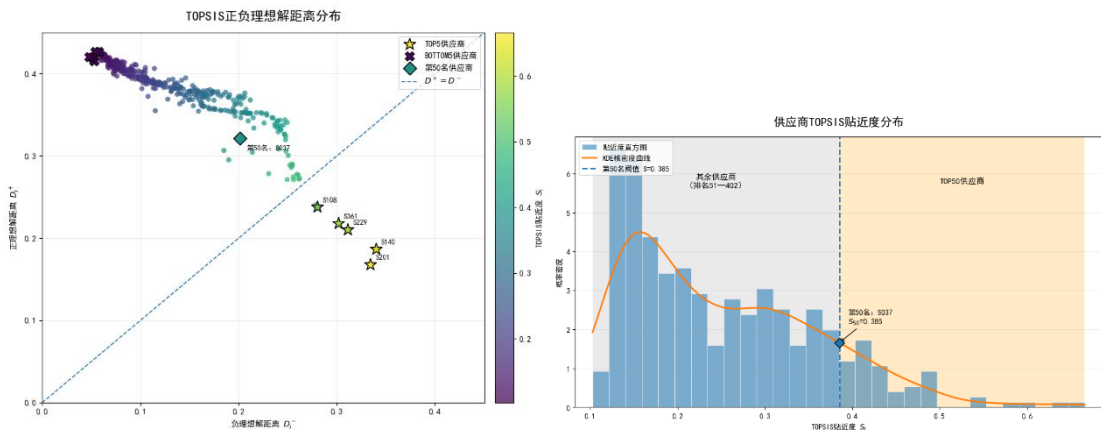


图 8 供应商 TOPSIS 评价结果图

据图 8 分析，TOPSIS 评价结果对供应商表现具有良好的区分能力，头部供应商的综合优势显著且稳健，可作为重点合作对象的筛选依据。我们最终得出排名前 50 的供应商如表 2。

表 2 排名前 50 的供应商

排名	供货商	排名	供货商	排名	供货商	排名	供货商
1	S201	14	S131	27	S226	40	S123
2	S140	15	S268	28	S364	41	S067
3	S229	16	S306	29	S374	42	S213
4	S361	17	S356	30	S218	43	S055
5	S108	18	S194	31	S294	44	S175
6	S151	19	S348	32	S076	45	S174
7	S282	20	S352	33	S367	46	S338
8	S340	21	S143	34	S080	47	S126
9	S308	22	S247	35	S346	48	S221
10	S275	23	S284	36	S098	49	S169
11	S329	24	S365	37	S307	50	S037
12	S330	25	S031	38	S007		
13	S139	26	S040	39	S244		

5.5 结果分析

为分析头部集中度与供应商分层，本文绘制出 Pareto 累计贡献曲线图和三类供应商指标对比雷达图，如图 9 所示。

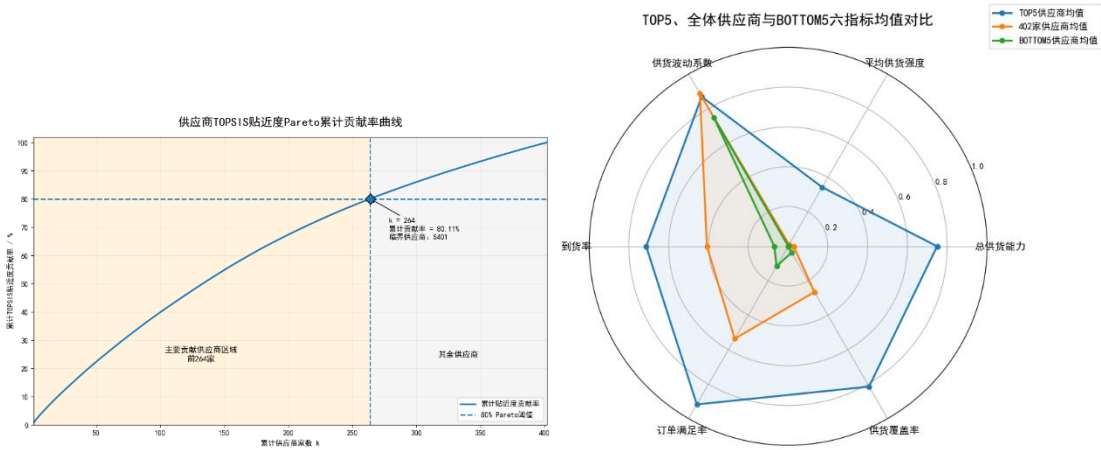


图 9 供应商评价分层分析图

由图 9 可知，优质供应商高度集中，符合“二八法则”。且通过进一步对比 TOP5、402 家均值与 BOTTOM5 的六项指标得分，进一步验证了该指标在评价体系中的弱区分能力。

为分析头部供应商的特征画像，本问输出了 TOP50 箱线图和三类供应商 TOPSIS 贴进度分布图，如图 10 所示。

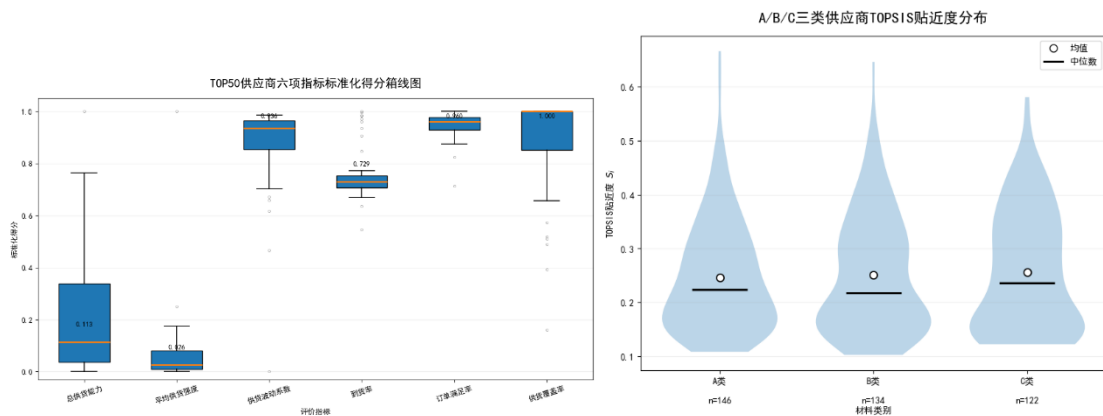


图 10 头部供应商特征图

据图 10 得出：头部供货商的规模存在较大的阶梯分化；“平均供货强度”是区分 TOP50 内部排名的重要维度；而波动性、到货率、订单满足率及覆盖率方面趋于一致；C 类供应商整体供货稳定性最优，但并非供应商综合表现的决定性因素。

六、 问题二的模型建立与求解

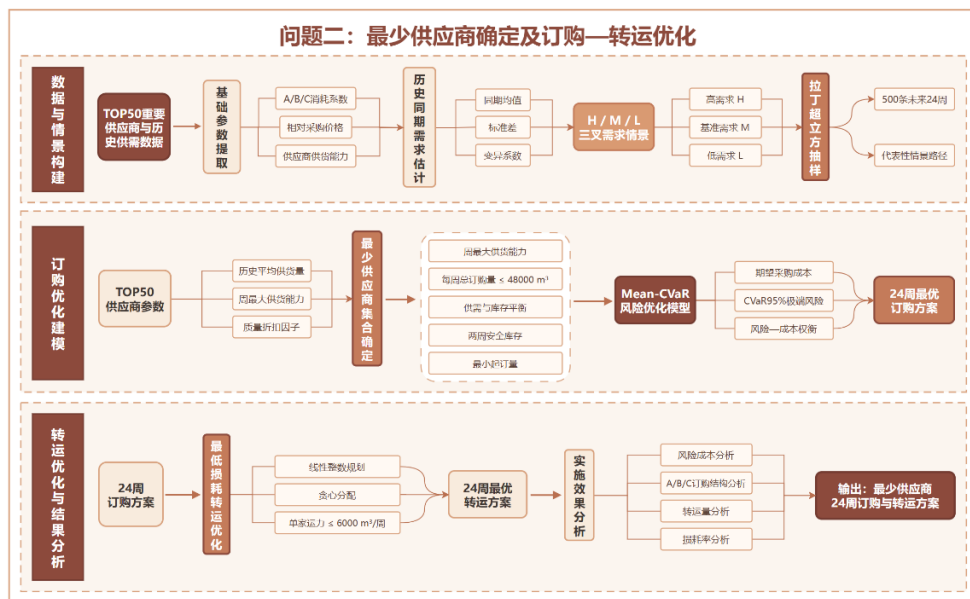


图 11 问题二求解思路图

6.1 基础数据提取与参数定义

基于问题一筛选出的重要供应商中提取出 m 类原材料， $m \in \{A, B, C\}$ ，供应商为 i 。本问定义相对价格系数 $P_A = 1.2, P_B = 1.1, P_C = 1.0$ ，由题得三类材料的消耗系数分别为 $\alpha_A = 0.6, \alpha_B = 0.66, \alpha_C = 0.72, D_m^* = 2.82x d m$ 。

本问先用历史同期均值法推算第 t 周对 m 类原材料的基准需求量：

$$D_{tm}^0 = \frac{1}{5} \sum_{y=0}^4 q_{(y \times 48 + t), m} \quad (21)$$

其中 q 为周订货量。

然后计算 240 周的标准差和变异系数：

$$\sigma_{tm} = \sqrt{\frac{1}{4} \sum_{y=0}^4 (q_{(y \times 48 + t), m} - D_{tm}^0)^2} \quad (22)$$

$$CV_{tm} = \frac{\sigma_{tm}}{D_{tm}^0} \quad (23)$$

输出表格见支撑材料。

6.2 情景树构建

为了刻画未来 24 周原材料需求的不确定性，构建三叉情景树^[5]。本问首先将每周需求分为高需求(H)、基准需求(M)、低需求(L)：

$$\text{每周需求} = \begin{cases} H: q_{tm} \geq D_{tm}^0 + 1.5\sigma_{tm} \\ M: D_{tm}^0 - 1.5\sigma_{tm} < q_{tm} < D_{tm}^0 + 1.5\sigma_{tm} \\ L: q_{tm} \leq D_{tm}^0 - 1.5\sigma_{tm} \end{cases} \quad (24)$$

分别统计 240 周中 A、B、C 三类材料的占比，得到情景概率为表 3 所示。

表 3 情景概率统计表

材料类型	高需求 pH	基准需求 pM	低需求 pL
A	0.1125	0.8667	0.0208
B	0.1125	0.8792	0.0083
C	0.0792	0.9042	0.0167
平均值（取整）	0.1000	0.8800	0.0200

其次，结合三类情景的需求系数 $\alpha_t^{(r)}$ ，其中 r 为后期路径：

$$\alpha_t^{(r)} = \begin{cases} 1 + 1.5 \times CV_{tm}, r \in H \\ 1, r \in M \\ 1 - 1.5 \times CV_{tm}, r \in L \end{cases} \quad (24)$$

接着，在未来的 24 周，本问采用拉丁超立方抽样^[6]从所有可能路径中抽取 500 条具有代表性的情景路径。第 r 条路径第 t 周对 m 材料的实际需求量为：

$$D_{tm}^{(r)} = \alpha_t^{(r)} \cdot D_{tm}^0, r = 1, 2, \dots, 500 \quad (25)$$

最后，输出情景树结构图与 500 条情景样本路径^[7]如图 12 所示。

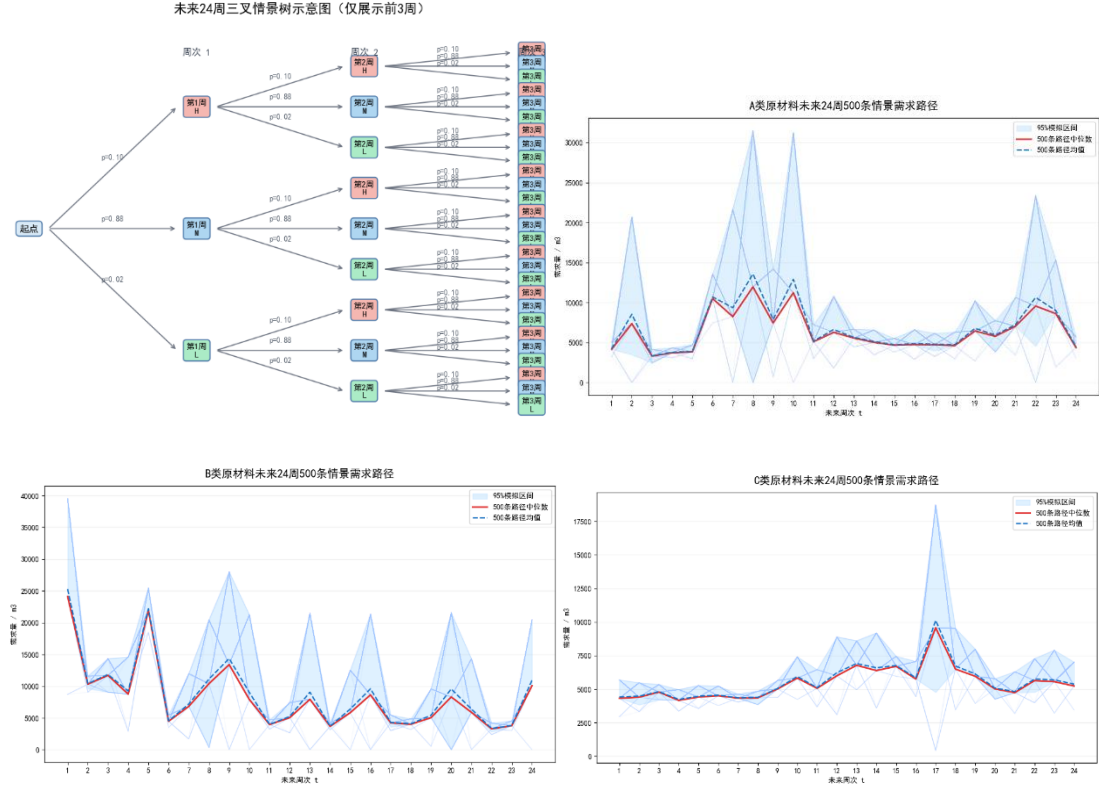


图 12 三叉情景树结构及原材料需求模拟路径分布图

根据图 12 可知：A 类原材料 500 条模拟路径的中位数与均值几乎重合，95%区间呈现明显时变特征，B、C 类结果类似。该组图为订购决策提供了完整的需求情景输入。

6.3 订购方案

6.3.1 决策变量

本问定义 $\bar{s}_{im}$ （万立方米）为历史平均周供货量； $\bar{s}_{im}^{max}$ （万立方米）为周最大供货能力； S_i 为贴进度； λ_i 为供应商质量折扣因子：

$$\lambda_i = \frac{1}{0.2 + 0.8S_i}, S_i \in [0,1] \text{ 使 } \lambda_i \in [1,5] \quad (26)$$

对于每个情景 $r \in R$ ， R 情景路径集合， $\|R\| = 500$ ； $o_{itm} \geq 0$ ：第 t 周向供应商 i 订购 m 类材料数量； $I_{tm}^{(r)+} \geq 0$ ：第 t 周末 m 类的库存量； $I_{tm}^{(r)-} \geq 0$ ：第 t 周末 m 类的缺货量； $y_{it} \in \{0,1\}$ ：第 t 周是否有供应商 i 订货； $u_i = \begin{cases} 1, \text{选择供应商 } i \\ 0, \text{不选择供应商 } i \end{cases}$ ；各转运商具体损耗率为 $l_k, k = 1, 2, \dots, 8$ ；全体转运商平均损耗率为 $\bar{l} = \frac{\sum s_{itk} l_k}{\sum l_k}$ ；算术平均损耗率为 $\bar{l}_k = \frac{1}{T_k} \sum_{t=1}^{T_k} l_{kt}$ ， T_k 为转运商 k 实际有损耗率的周次集合。目标函数为：

$$\min \sum_i u_i \quad (27)$$

6.3.2 约束条件

本问设置约束条件为：周最大供货能力：

$$o_{itm}^{(r)} \leq s_{im}^{-max} \cdot y_{it}^{(r)} \quad (28)$$

每周总订购量运力约束：

$$\sum_{i=1}^{50} \sum_{m=1}^3 o_{itm} \leq 48000 \quad (29)$$

供需平衡：其中初始库存 $I_{0,m}^{(r)+} = 0$

$$\sum_{i=1}^{50} o_{itm} \cdot (1 - \bar{l}) + I_{t-1,m}^{(r)+} - I_{tm}^{(r)+} + I_{tm}^{(r)-} = D_{tm}^{(r)} \quad (30)$$

安全库存：其中 B_{tm} 为基准状态下用于检查两周安全库存的库存量； V_{tm} 为安全库存目标的不足量

$$B_{tm} + V_{tm} \geq 2\bar{D}_{tm} \quad (31)$$

情景库存平衡： Inv_{rtm} 为第 r 种未来情况真的发生时，第 t 周生产后的实际库存； $Short_{rtm}$ 为该情景下已将库存全部用完后的真实缺货； z_{tm} 为第 t 周企业实际收到的 m 类原材料到货量； I_m 为所有供应 m 类原材料的供应商集合

$$Inv_{r,t-1,m} + z_{tm} + Short_{rtm} = D_{rtm} + Inv_{rtm} \quad (32)$$

$$z_{tm} = (1 - i) \sum_{i \in I_m} o_{it} \quad (33)$$

供应商范围： $i \in \{\text{问题一确定的 50 家重要供应商}\}$

最小起订量：其中 q_{im}^{min} 为 i 对 m 的历史最小正供货量

$$o_{itm}^{(r)} \geq q_{im}^{min} \cdot y_{it}^{(r)} \quad (34)$$

相对价格系数： $P_A = 1.2, P_B = 1.1, P_C = 1$

6.3.3 情景成本函数

根据实际经验，本文定义库存持有成本参数为 $h_m = 0.3 \times P_m$ ，缺货惩罚系数为 $g_m = 5 \times P_m$ ，故第 r 个情景下的总成本为：

$$C^{(r)} = \sum_{t=1}^{24} \sum_{m=A}^c \sum_{i=1}^{50} P_m \cdot \lambda_i \cdot o_{itm}^{(r)} + \sum_{t=1}^{24} \sum_{m=A}^c h_m \cdot I_{tm}^{(r)+} + \sum_{t=1}^{24} \sum_{m=A}^c g_m \cdot I_{tm}^{(r)-} \quad (35)$$

6.3.4 风险度量(Mean-CVaR)

本问首先将 500 个情景成本 $C^{(r)}$ 从高到低排序，取最高的 5%的平均值为 $CVaR_{95\%}$ ，进行线性化处理，最后得到完整目标函数。

基于上述步骤^[8]，输出了最少供应商数量及订购方案，具体见支撑材料。同时，输出不同风险偏好下情景成分分布小提琴图+箱线图，未来 24 周原材料订货量的堆叠柱状图以及供应商累计订货量的水平条形图，结果如图 13 所示。

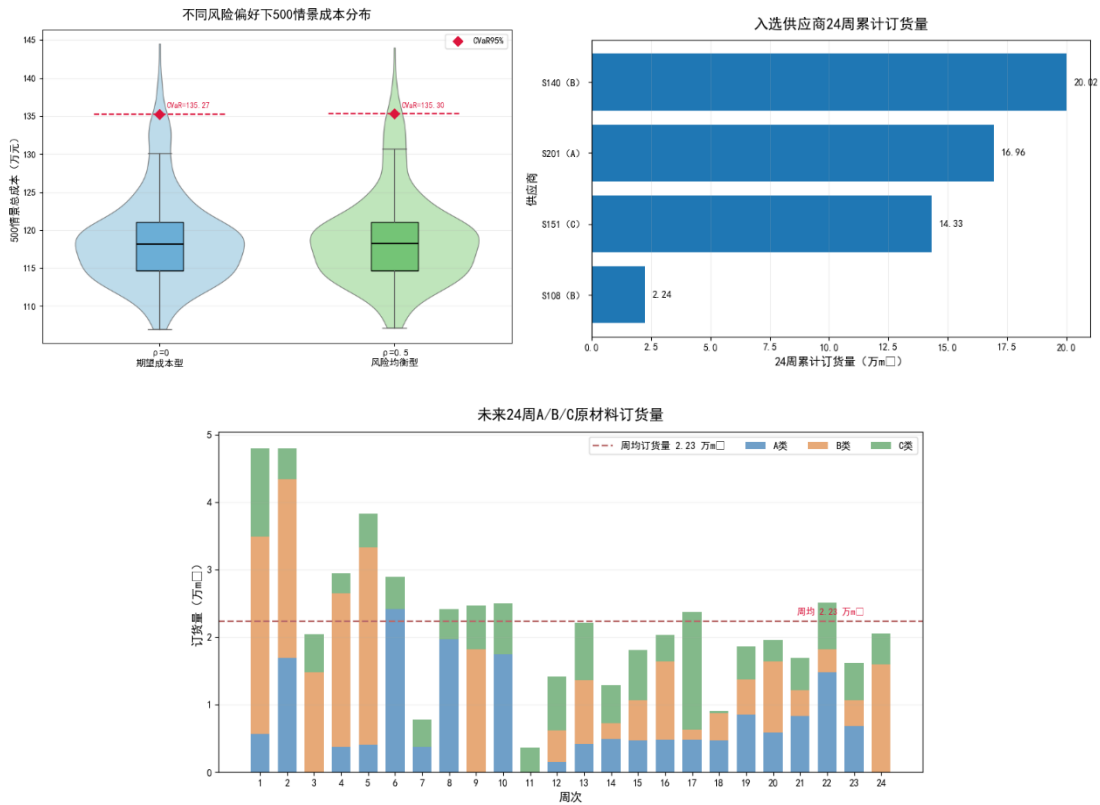


图 13 订购优化方案结果分析图

根据图 13 分析可知：均衡型方案能以极小成本代价有效控制极端风险；模型会依据需求波动动态调整订购节奏，有效避免库存积压；订单集中于质量折扣因子更优的供应商，与上文综合评价结果相同。综上，本模型在成本与风险间取得有效平衡，订购方案具备需求响应能力与供应商择优分配能力。

6.4 转运方案

已知每家转运商每周运力为 $Cap_k \leq 6000$ ；第 t 周供应商 i 的供货量是否分配给转运商 k ， $w_{itk} = \begin{cases} 0, & \text{不分配} \\ 1, & \text{分配} \end{cases}$ 且 $\sum_{k=1}^8 s_{itk} = Q_{it}$ ；转运商运力 $\sum_i s_{itk} \leq 6000$ ，其中第 t 周供应商 i 的总供货量 $Q_{it} = \sum_m o_{itm}$ ， $s_{itk} \leq 6000w_{itk}$ 。

在给定的订购方案下，运输损耗基本不变且考虑到单家运力不足时，按损耗率从低到高依次进行补充，即在极端情况下采取**多家运输**，从而确定了目标函数：

$$\min_{w_{itk}} \sum_{t=1}^{24} \sum_{i=1}^{50} \sum_{k=1}^8 l_k \cdot s_{itk} \quad (36)$$

我们运用线性整数规划^[9]及贪心算法^[10]，最终得出最低损耗转运方案，具体见支撑材料。同时，输出转运商运输量分组柱状图+损耗率叠加折线图和供应商-原材料-转运桑基图，如图 14 所示。

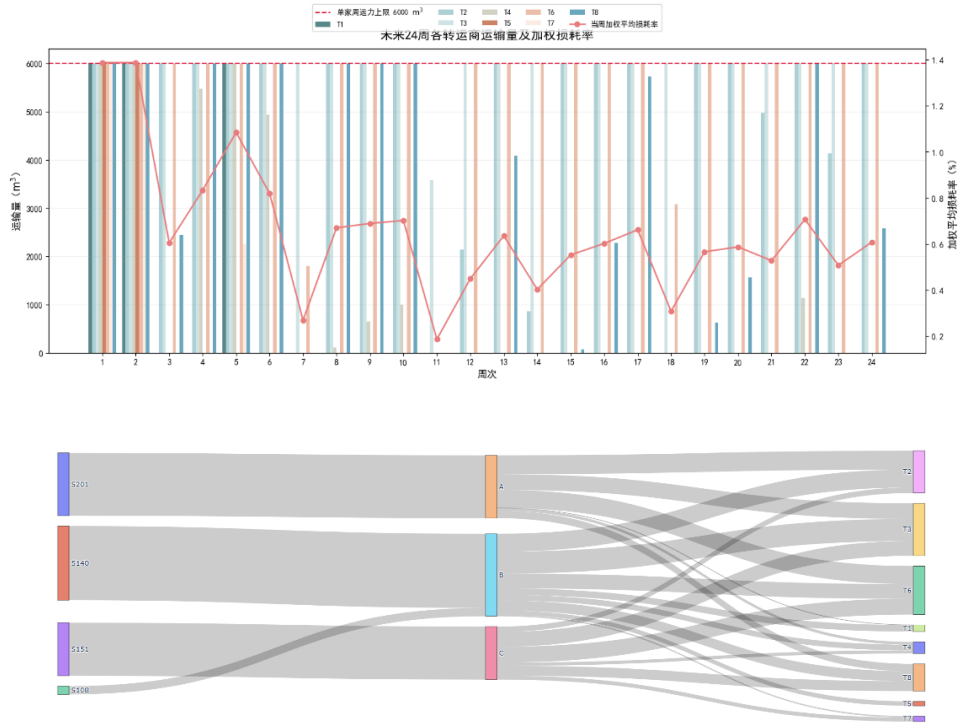


图 14 转运方案优化结果图

由图 14 可知，优化后的多路径转运方案成功避免了运力超出风险，全周期损耗率始终保持在 1.4% 以下，证明模型在满足硬约束的同时兼具极强的经济效益。

七、 问题三的模型建立与求解

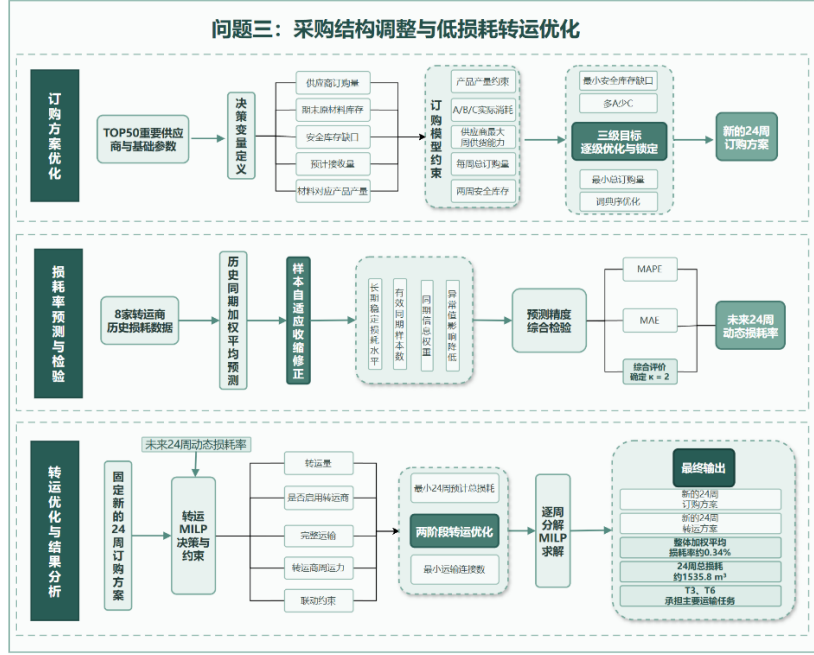


图 15 问题三求解思路图

7.1 订购方案

本问设 i 为问题一的 TOP50 重要供应商。 $t = 1, 2, \dots, 24$ 为未来 24 周。

7.1.1 决策变量

在本问中 $o_{it} \geq 0$ 为订购量； $I_{tm} \geq 0$ 为第 t 周末 m 类原材料库存； $V_t \geq 0$ 为第 t 周距离两周安全库存目标仍不足的等价产品量； $\bar{l}$ 为历史平均损耗率；

$R_{tm} = (1 - \bar{l}) \sum_{i \in I_m} o_{it}$ 为材料 m 在第 t 周的预计接收量。

7.1.2 约束条件

本问设置约束条件为：产品产量：

$$Z_{tA} + Z_{tB} + Z_{tC} = 28200 \quad (37)$$

原材料实际消耗为：

$$V_{tA} = 0.6Z_{tA}, V_{tB} = 0.66Z_{tB}, V_{tC} = 0.72Z_{tC} \quad (38)$$

最大周供货量： $0 \leq o_{it} \leq s_i^{max}$ ；每周总转运能力： $\sum_i o_{it} \leq 48000$

两周安全库存：将三类库存全部折算为能够支持的等价产品产量

$$\frac{I_{tA}}{0.6} + \frac{I_{tB}}{0.66} + \frac{I_{tC}}{0.72} + V_t \geq 56400 \quad (39)$$

7.1.3 目标函数

本问由

$$\min V = \sum_{t=1}^{24} V_t \quad (40)$$

得到 V^* ，且锁定 $\sum_t V_t \leq V^* + \varepsilon_V$ 。在保持 V^* 的前提下：

$$\min J_{AC} = \sum_{t=1}^{24} (Z_{tC} - Z_{tA}) \quad (41)$$

得到 J_{AC}^* ，且锁定 $\sum_t (Z_{tC} - Z_{tA}) \leq J_{AC}^* + \varepsilon_{AC}$ 。在上两个条件下：

$$\min F = \sum_t o_{it} \quad (42)$$

得到 F^* ，且锁定 $\sum_{i,t} o_{it} \leq F^* + \varepsilon_F$ 。

7.1.4 结果分析

基于上述步骤，运用字典序优化^[11]最终输出订购方案，具体见支撑材料。同时，输出问题二和问题三 A、B、C 采购的对比旭日图以及每周库存变化与安全目标的折线图+面积图，如图 16 所示。

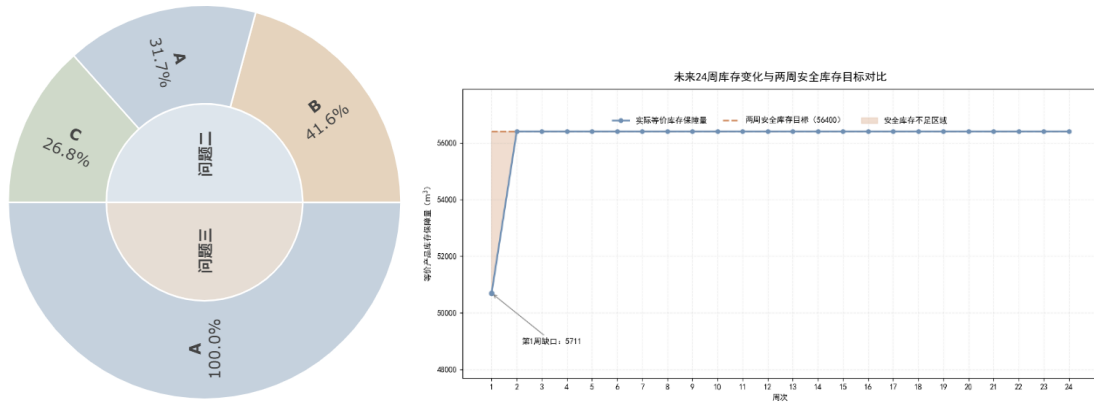


图 16 优化方案下原材料采购结构变化与库存保障效果

图 16 中结果表明，优化方案使采购结构发生较大变动，A 类原材料占比由 31.7%增至 100.0%。库存动态虽在第 1 周存在 5,711 m³的初始缺口，但第 2 周即回升并稳定达标，在后续周期内零波动，验证了策略的强稳定性。

7.2 转运方案

7.2.1 转运损耗率预测

本问设定 c 为 8 家转运商未来 24 周的预测损耗率，用历史同期加权平均预

测， $Y = 1, 2, 3, 4, 5$ (表示 1~5 年)， $\omega_Y = 6 - Y$ (对应年份权重)：

$$\hat{\lambda}_{kt} = \frac{\sum_{Y=1}^5 \omega_Y \cdot \lambda_{k,t-48Y}}{\sum_{Y=1}^5 \omega_Y} \quad (43)$$

7.2.2 样本自适应收缩修正

为避免单个异常损耗值对预测结果产生较大影响，本问引入 b_k 为转运商 k 的长期稳定损耗水平； $n_{k,t} = |y_{k,t}|$ 未来第 h 周转运商 k 的有效同期样本数量为： $\delta_{k,t} = \frac{n_{k,t}}{n_{k,t} + \kappa}$ 为同期信息权重， $\kappa > 0$ 为收缩参数。最终预测损耗率为：

$$\hat{\lambda}_{kt} = \delta_{k,t} \tilde{\lambda}_{k,t} + (1 - \delta_{k,t}) b_k \quad (44)$$

输出双指标自适应收缩结果，具体见支撑材料。同时，得出双指标自适应收缩前后预测精度对比图和未来 24 周转运损耗率，如图 17 所示。

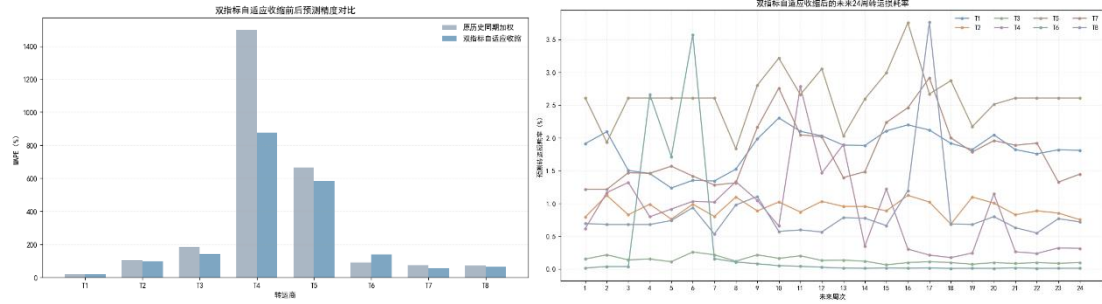


图 17 双指标自适应收缩模型预测性能评估图

图 17 表明，双指标自适应收缩方法能有效避免历史加权法的异常高估，并准确区分高损耗与平稳线路，为转运环节提供了可靠的决策依据。

7.2.3 验证预测精度

本问采用平均绝对百分比误差 MAPE 和平均绝对误差 MAE 综合评价预测效果：

$$MAPE = \frac{1}{n} \sum \left| \frac{\hat{\lambda}_{kt} - \lambda_{kt}}{\lambda_{kt}} \right| \times 100\% \quad (45)$$

$$MAE = \frac{1}{n} \sum |\hat{\lambda}_{kt} - \lambda_{kt}| \quad (46)$$

经计算我们得出：MAPE = 115.80%，MAE = 0.4478，且本文最终取 $\kappa = 2$ 。当 $K=2$ 时，总体 MAPE 约 90.95%，MAE 约 0.7984 个百分点，MAPE 与 MAE 的综合评价达到最优。

7.2.4 目标函数

本问先设定决策变量： $s_{itk} \geq 0$ 为供货量； $c \in \{0,1\}$ 为是否分配给转运商 k 。其次，明确约束条件：**完整运输**： $\sum_{k=1}^8 s_{itk} = o_{it}$ ；**转运商运力**： $\sum_i s_{itk} \leq 6000$ ； $s_{itk} \leq 6000w_{itk}$ 。接着，设定目标函数：

$$\min \mathcal{A} = \sum_{t=1}^{24} \sum_i \sum_{k=1}^8 s_{itk} \hat{\lambda}_{kt} \tag{47}$$

得到 $\mathcal{A}^*$ 且锁定 $\mathcal{A} \leq \mathcal{A}^* + \varepsilon$ ，然后

$$\min \sum_{t=1}^{24} \sum_i \sum_{k=1}^8 w_{itk} \tag{48}$$

最后，输出逐周分解转运 MILP^[12]最终结果，具体见支撑材料。同时，得出 8 家转运商 24 周汇总表和每周转运商运力利用率堆叠面积图，如表 4 和图 18 所示。

表 4 八家转运商 24 周汇总表

转运商	24 周累计转 运量	24 周预计损 耗量	平均周运力 利用率	最大周运力 利用率	负荷加权平 均损耗率
T3	143999.8779	193.4626	99.9999	100.0	0.1343
T6	125999.9099	40.9378	87.4999	100.0	0.0325
T8	76423.7723	541.5252	53.0721	100.0	0.7086
T4	57581.9452	202.6439	39.9875	100.0	0.3519
T2	24107.3096	213.0567	16.7412	100.0	0.8838
T1	6000.0000	114.7883	4.1667	100.0	1.9131
T5	6000.0000	156.5220	4.1667	100.0	2.6087
T7	6000.0000	72.8766	4.1667	100.0	1.2146

据表 4 可知：T3 和 T6 损耗率显著低于平均水平，说明模型优先使用低损耗转运商。T1、T5、T7 损耗率较高仅作应急备用。所有转运商单周最大运量均未超过 6000 立方米，运力约束满足；整体加权平均损耗率 0.34%，24 周总损耗 1535.8 立方米。

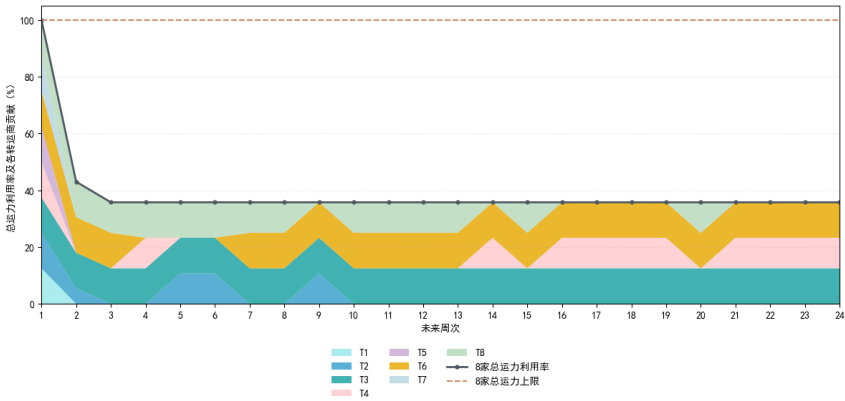


图 18 未来 24 周转运商运力利用率堆叠面积图

据图 18 可知：第 1 周总运力利用率达 100%，原因是首周需同时满足生产和建立两周安全库存的需求。第 2 周起总运力利用率稳定在 35%-40%，说明后续运输需求平稳。各转运商中，T3 和 T6 承担主要运输任务，T1、T5、T7 仅在首周被调用，与上表结论一致。

八、 问题四的模型建立与求解

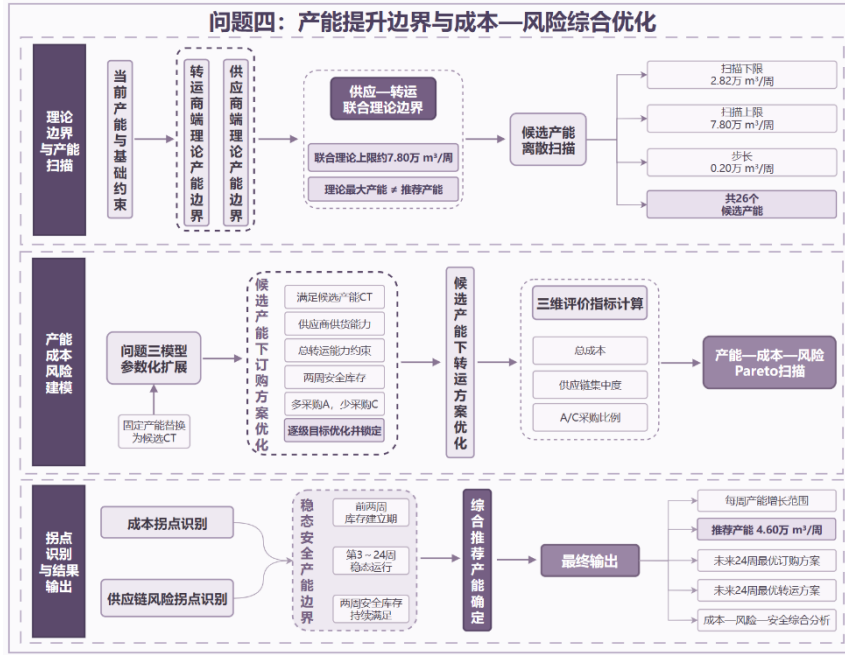


图 19 问题四求解思路图

8.1 产能提升的理论边界约束

转运商运力：已知 8 家 k 总运力为 48000 立方米每周，定义 x_m 为每天实际运输多少 m 类原材料，则转运商运力约束下的最大产能为：

$$CT = (1 - \bar{\lambda}) \left(\frac{x_A}{0.6} + \frac{x_B}{0.66} + \frac{x_C}{0.72} \right) \quad (49)$$

当采购结构偏向 A 类（ ξ 趋近于 0.6）时产能上限较高，当采购结构偏向 C 类（ ξ 趋近于 0.72）时产能上限较低，则转运商端产能限范围为 65800~78900 立方米每周。

供应端：三类材料在历史上各自的最大周聚合供货量，决定了原材料端能支撑的产能上限。本问设第 t 周第 m 类材料的周聚合供货量为： $s_m(t) = \sum_{i \in I_m} s_i(t)$ ， I_m 为供应 m 类材料的供应商集合， $s_m(t)$ 为供应商 i 在第 t 周的供货量；各类材料极限周供应能力为：

$$s_m^{max} = \max_{t \in [1, 240]} s_m(t) \quad (50)$$

折算为产能上限：

$$CT_m^{sup} = \frac{s_m^{max} \times (1 - \bar{l})}{\xi_m} \quad (51)$$

三类材料必须同时满足，

$$x_A + x_B + x_C \leq 48000, x_A \leq 42075, x_B \leq 3525 \quad (52)$$

基于以上步骤，输出产能理论边界分析，具体见支撑材料。接着，本文对CT的范围进行扫描，输出最大可支撑产能等高线图，如图20所示。

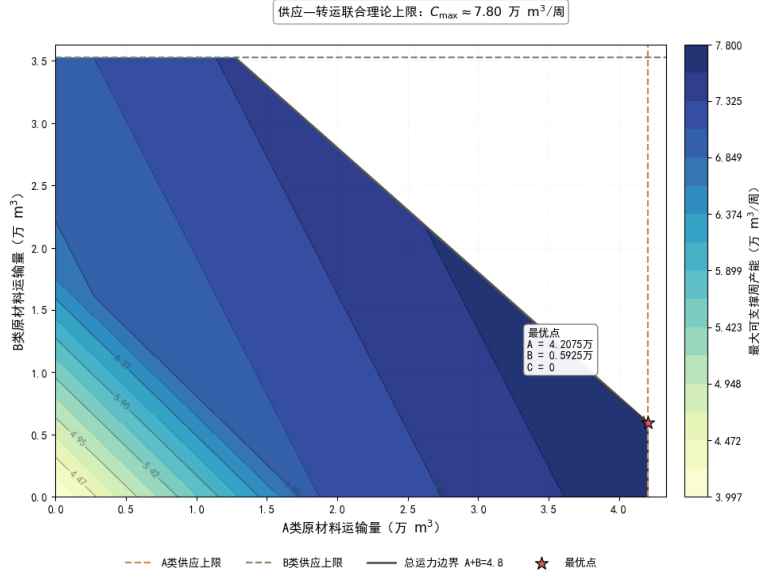


图 20 A、B 原材料配置下的最大可支撑产能等高线图

当前基准产能为2.82万，扫描上限为7.8，最终扫描范围设为[2.82,7.8]，步长为0.2，候选值有26个，具体见支撑材料。

8.2 产能-成本-风险帕累托分析模型

本模型是对问题三模型参数化拓展。本问将问题三模型中的固定产能参数2.82万立方米每周替换为决策变量CT，对每个候选产能值 $CT \in \{求解出来的26个, 2.82, 3.00, 3.20, 3.40, \dots, 7.80\}$ 。

订购方案：本问在问题三的基础上，新加入决策变量CT，并将其参数化。目标函数：

$$\min f_0 = \sum_{t=1}^{24} V_t \quad (53)$$

得到 f_0^* ，并锁定 $\sum_{t=1}^{24} V_t \leq V^* + \varepsilon_V$ ：

$$\min f_1 = \sum_{t=1}^{24} \sum_{i \in I_C} o_{it} c \quad (54)$$

得到 f_1^* ，并锁定 $f_1 = f_1^*$ ：

$$\min f_2 = \sum_{t=1}^{24} \sum_{i \in I_A} o_{itA} \quad (55)$$

得到 f_2^* ，并锁定 $f_2 = f_2^*$ ：

$$\min f_3 = \sum_{t=1}^{24} \sum_m P_m \sum_{i \in I_m} o_{itm} \quad (56)$$

其中， P_m 为前面的相对价格系数。

$$\min f_4 = \sum_{t=1}^{24} \sum_m P_m I_{tm} \quad (57)$$

转运方案：本问损耗率预测、决策变量和约束条件均与问题三一致，目标函数为：

$$\min \sum_{t=1}^{24} \sum_i \sum_{k=1}^8 Z_{itk} \cdot \bar{l}_{kt} \cdot s_{it} \quad (58)$$

三个评价维度：对每个候选产能值 CT，求解上述模型后记录以下指标：

总成本：

$$\begin{aligned} C(CT) = & \sum_{t=1}^{24} \sum_m P_m \sum_i o_{itm} + \sum_{t=1}^{24} \sum_i \sum_{k=1}^8 Z_{itk} \cdot \bar{l}_{kt} \cdot s_{it} \\ & + h \sum_{t=1}^{24} \sum_m I_t^m C(CT) \end{aligned} \quad (59)$$

供应链集中度：采用赫芬达尔·赫希曼指数 HHI (CT)

$$HHI(CT) = \sum_{i=1}^N \left(\frac{\sum_{t=1}^{24} s_{it}}{\sum_{t=1}^{24} \sum_j s_{jt}} \right)^2 \times 10000 \quad (60)$$

其中，N 为实际启用的供应商数量。HHI 越高说明供应商集中度越高，供应链风险越大。按经济学标准： $HHI < 1500$ 为竞争型， $1500 \leq HHI \leq 2500$ 为适度集中， $HHI > 2500$ 为高度集中。

A/C 采购比例 $R_{A/C}$ ：

$$R_{A/C}(CT) = \frac{\sum_t \sum_{i \in I_A} o_{itA}}{\sum_t \sum_{i \in I_C} o_{itC}} \quad (61)$$

最后输出产能成本风险 Pareto^[13]扫描结果，具体见支撑材料。并绘制总成本与供应链集中度变化图和成本-供应链风险 Pareto 候选前沿图，如图 21 所示。

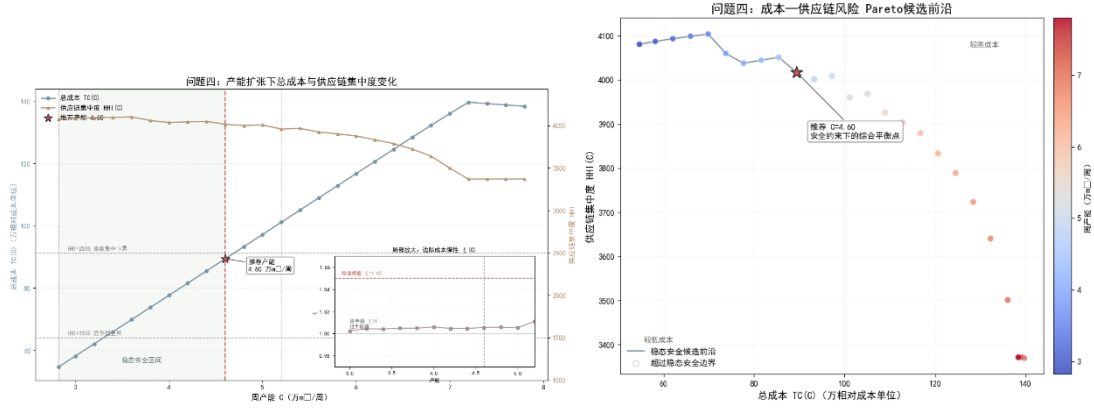


图 21 产能扩张全局寻优与稳态安全约束下的推荐方案

由图 21 可知：随产能增加，总成本持续上升；前期供应链集中度平缓，后期显著下降。C=4.60 处边际成本弹性处于安全区间，成本增速可控。右图点位于 Pareto 前沿拐点，完美权衡了成本与风险。

8.3 拐点识别与推荐产能

对 26 个候选产能值逐个求解后，得到产能-成本-风险三维数据。在此基础上，用两个定量标准识别拐点。

成本拐点（边际成本弹性法）：定义边际成本弹性：

$$\varepsilon(CT) = \frac{\frac{\Delta T_C}{T_C}}{\frac{\Delta CT}{CT}} = \frac{\Delta T_C}{\Delta C} \cdot \frac{C}{T_C} \quad (62)$$

当 $\varepsilon(CT) > 1.05$ 且两个连续候选档都成立时，总成本增速超过产能增速，即成本进入加速上升区间。

成本拐点定义为：

$$CT_{cost} = \min\{CT | \varepsilon(CT) > 1\} \quad (63)$$

风险拐点（HHI 变化率法）：定义 HHI 变化率：

$$\rho(CT) = \frac{HHI(CT) - HHI(CT - \Delta CT)}{HHI(CT - \Delta CT)} \quad (64)$$

稳态安全边界为 CT_{safe} ，当 $\rho(CT) > \eta$ 且 $CT \leq CT_{safe}$ ，本问取 $\eta = 0.15$ ，即 HHI 单步增幅超过 15% 时，供应商集中度开始急剧上升。

风险拐点定义为：

$$CT_{risk} = \min\{CT | \rho(CT) > \eta\} \quad (65)$$

推荐产能：在帕累托前沿上，推荐产能取两个拐点中较小值：

$$CT_{re} = \min\{CT_{cost}, CT_{risk}, CT_{safe}\} \quad (66)$$

基于以上步骤，输出了产能扩张的成本-风险-安全综合分析图，如图 22 所示。

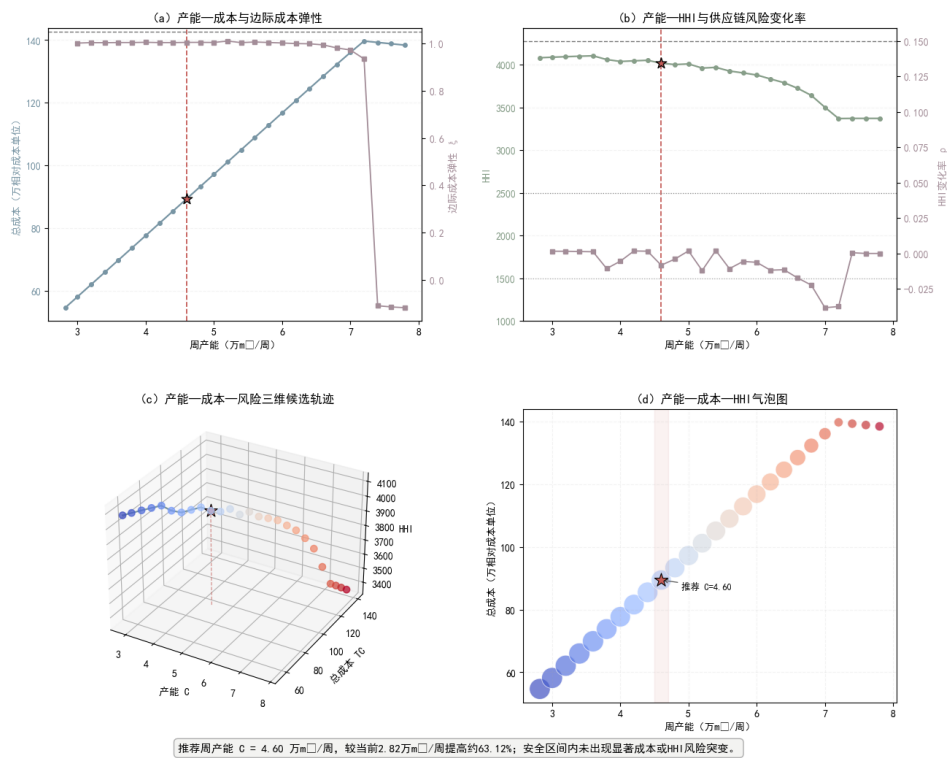


图 22 产能扩张的成本-风险-安全综合分析图

由图得出结论：

成本效益分析：总成本随产能增加呈近似线性上升，但在 $C=4.60$ 万 m^3 /周处，边际成本弹性出现显著拐点，表明进一步扩产的成本增速将大幅放缓，具备规模化经济潜力。

供应链风险演进：产能扩张初期 HHI 指数保持高位平稳，供应链高度集中；当扩产至 $C=4.60$ 后，HHI 开始呈现下降趋势，且风险变化率在安全阈值内波动，未出现断崖式突变，扩产行为未引发实质性风险集聚。

多维耦合决策： $C=4.60$ 万 m^3 /周处于成本、风险与安全边界的最优耦合点。最终推荐产能为 $C=4.60$ 万 m^3 /周。该方案较当前产能提升约 63.12%，且此区间内未出现显著的成本突变或 HHI 风险失控，实现了产能效益与供应链安全的协同最优。

九、模型的分析与检验

9.1 模型误差分析

为检验模型在数据处理、情景抽样、预测及参数离散化过程中产生的误差，本文分别从问题一异常值处理、问题二蒙特卡洛情景抽样、问题三转运损耗率预测以及问题四产能扫描四个方面进行误差检验，结果如图 23 所示。

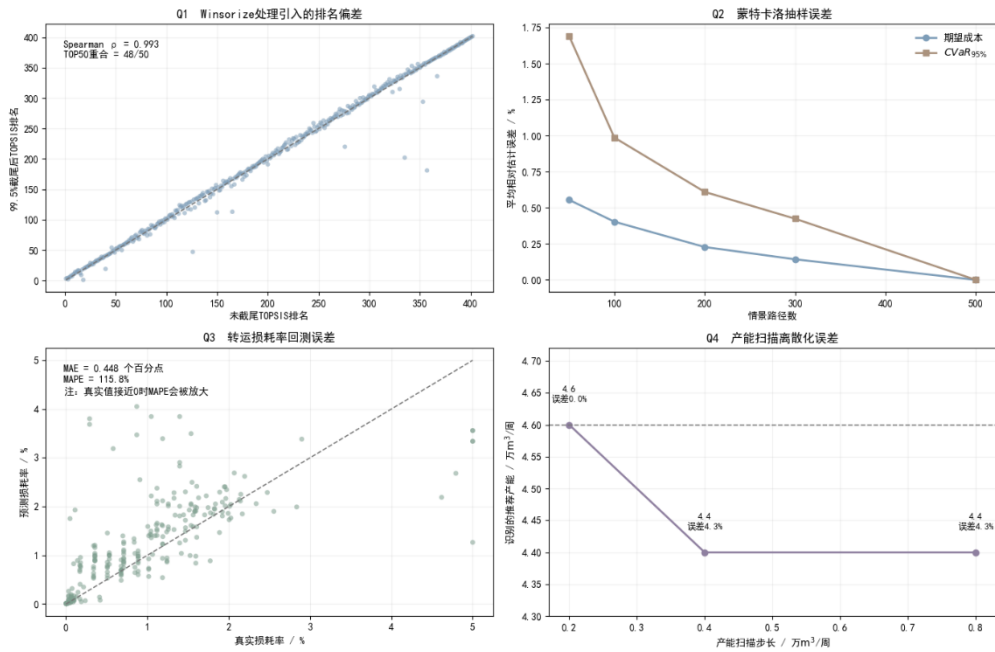


图 23 四个问题误差分析图

由图 23 得出结论：问题一至问题四中的异常值处理误差、随机抽样误差、预测误差及离散化误差均未造成核心模型结论的根本改变，模型整体误差处于可控范围。

9.2 灵敏度分析

为考察关键参数变化对模型结果的影响，本文分别选取问题一的 Winsorize 截尾分位点、问题二 Mean-CVaR 模型中的风险权重、问题三自适应收缩参数以及问题四候选产能进行灵敏度分析，结果如图 24 所示。

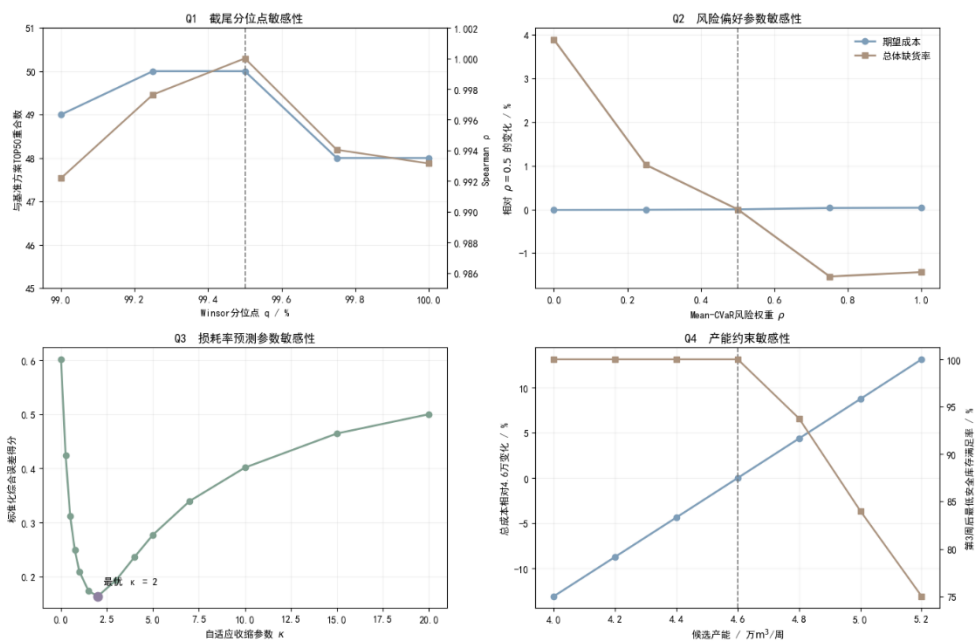


图 24 关键参数灵敏度分析图

由图 24 可知：问题一、二、三中的关键参数在合理区间变化时均未引起评价结构或优化方案的剧烈改变；问题四则在 4.60 万立方米每周附近呈现清晰的安全库存临界特征，进一步验证了推荐产能选择的合理性。

9.3 稳定性分析

为检验模型核心结论是否会因评价方法、风险偏好、预测方法或安全判断发生改变，本文分别从供应商排序方法、订购方案结构、转运商优先级以及推荐产能四个方面进行稳定性检验，结果如图 25 所示。

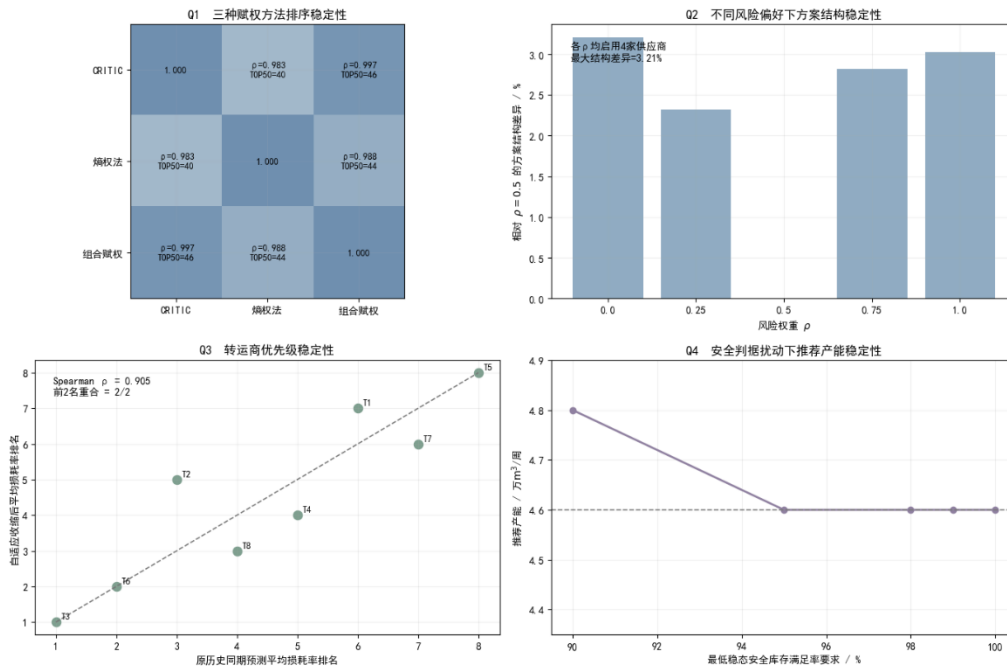


图 25 稳定性分析图

由图 25 得，供应商评价结果、问题二订购方案结构、问题三转运商优先级以及问题四推荐产能在合理的方法或参数扰动下均保持较稳定状态，说明本文模型的核心结论具有较好的稳定性与鲁棒性。

十、 模型的评价

10.1 模型优点

1. 构建了从完整供应链优化体系：本文各模型之间逻辑连贯，能够完整描述“供应商评价-订购-转运-扩产”这一实际生产决策过程。

2. 供应商评价采取组合赋权，减弱单一方法偏差：问题一将 CRITIC 方法与熵权法进行组合，再利用 TOPSIS 排序，使评价体系既考虑指标自身信息

量，又减少单一赋权方法的系统性偏好。

3. **将需求随机性和尾部风险纳入订购模型，具有较强风险控制能力：**问题二利用三叉情景和 500 条拉丁超立方抽样路径并结合 Mean-CVaR。这使订购方案不仅追求平均意义下的低成本，也能够避免极端需求情景造成严重缺货。

4. **转运模型兼顾损耗预测和真实运力：**问题三在历史同期预测的基础上加入样本自适应收缩，并将预测值进一步输入逐周 MILP 转运模型，不仅在目标函数上降低损耗，同时能够形成可直接执行的运输分配方案。

5. **产能推荐兼顾经济性、供应链风险与库存安全，而非单纯追求最大产能：**问题四未直接取理论最大值，而是形成多维评价体系，确定推荐产能。因此推荐值具有明确的成本-风险权衡意义，更符合企业扩产中的实际决策逻辑。

10.2 模型缺点

1. **未来数据主要依据历史数据刻画：**模型假设未来 24 周与过去 5 年的统计特征具有较强延续性，但历史参数未必能完全反映未来真实数据。

2. **转运损耗预测受到有效历史样本数量限制：**尽管本文使用样本自适应收缩和 MAE、MAPE 双指标进行修正和验证，但未来损耗预测仍存在不确定性。

3. **未考虑长期合作关系和突发供应链事件：**现有模型主要进行量化决策，没有进一步考虑低概率高影响事件，也未引入等实际管理因素。

10.3 模型改进

1. **引入滚动优化机制：**实际生产过程中可每周获得新的数据，因此可以将当前 24 周静态规划改造成“滚动预测-滚动优化”的模型。每经过一周重新更新未来需求和供应商能力，使方案能够动态适应市场变化。

2. **将供应商和运输商可靠性纳入鲁棒优化：**可进一步设置供应商停供概率、转运商失效概率和运输能力扰动区间，采用鲁棒优化或分布鲁棒优化构建能够抵抗突发供应链中断的采购运输方案。

十一、 AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、代码调试、细节完善等，详细使用情况见支撑材料。

十二、 参考文献

- [1] Diakoulaki D, Mavrotas G, Papayannakis L. Determining objective weights in multiple criteria problems: The CRITIC method[J]. Computers &

- Operations Research, 1995, 22(7): 763-770.
- [2] Shannon C E. A mathematical theory of communication[J]. Bell System Technical Journal, 1948, 27: 379-423, 623-656.
 - [3] Zheng G H, Li Y, Guo S M. Supplier selection based on the combination of entropy weight and TOPSIS[C]. DOI: 10.1061/40932(246)483.
 - [4] Hwang C L, Yoon K. Multiple Attribute Decision Making: Methods and Applications[M]. Berlin: Springer-Verlag, 1981.
 - [5] Maggioni F, Potra F A, Bertocchi M. A scenario-based framework for supply planning under uncertainty[J]. Computational Management Science, 2017, 14: 5-44.
 - [6] McKay M D, Beckman R J, Conover W J. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code[J]. Technometrics, 1979, 21(2): 239-245.
 - [7] Stein M. Large sample properties of simulations using Latin hypercube sampling[J]. Technometrics, 1987, 29(2): 143-151.
 - [8] Rockafellar R T, Uryasev S. Optimization of conditional value-at-risk[J]. Journal of Risk, 2000, 2(3): 21-41.
 - [9] 《运筹学》教材编写组编. 运筹学 第5版[M]. 北京: 清华大学出版社, 2021.10.
 - [10] Edmonds J. Matroids and the greedy algorithm[J]. Mathematical Programming, 1971, 1: 127-136.
 - [11] 汪应洛主编. 系统工程理论、方法与应用 第2版[M]. 北京: 高等教育出版社, 1998.05.
 - [12] Talluri S, Baker R C. A multi-phase mathematical programming approach for effective supply chain management[J]. International Journal of Production Research, 2002, 40(12): 2621-2638.
 - [13] Guerrero W J, et al. Simulation-optimization techniques for closed-loop supply chain design[J]. DYNA, 2018, 85(206).

附录

表 1 支撑文件目录

支撑文件名称	文件内容
AI 工具使用详情.pdf	AI 工具使用详情
问题一数据预处理结果.xlsx	问题一数据预处理结果
问题一输出结果图.docx	问题一：输出数据及分析
最少供应商及订购方案.xlsx	问题二：最少供应商数据及订购方案
最低损耗转运方案及数据.xlsx	问题二：最低损耗转运方案及数据
问题二基准需求与波动特征.xlsx	问题二：基准需求与波动特征
问题二输出结果图.docx	问题二：输出结果及分析
多 A 少 C 订购层最终结果.xlsx	问题三：订购方案的最终结果
问题三输出结果图.docx	问题三：输出结果及分析
双指标自适应收缩结果.xlsx	问题三：损耗率预测及自适应收缩结果
逐周分解转运 MILP 最终结果.xlsx	问题三：逐周分解转运 MILP 最终结果
问题四输出结果图.docx	问题四：输出结果及分析
产能理论边界分析.xlsx	问题四：产能理论边界分析
产能成本风险 Pareto 扫描结果.xlsx	问题四：产能成本风险 Pareto 扫描结果
附件 A 订购方案数据结果.xlsx	问题二到四：订购方案数据结果
附件 B 转运方案数据结果.xlsx	问题二到四：转运方案数据结果
完整代码.ipynb	题目完整代码

表 2 附录目录

附录	名称
附录 1	供需偏差稳健处理与六维供应商指标构核心代码
附录 2	CRITIC-熵权组合赋权核心代码
附录 3	三状态随机需求建模与 Monte Carlo 情景生成核心代码
附录 4	Mean-CVaR 随机订购优化代码
附录 5	采购结构调整的四阶段词典序线性规划核心代码
附录 6	自适应损耗率预测与动态两阶段转运优化核心代码
附录 7	联合产能边界与多目标产能扫描决策核心代码

附录 1

介绍：该代码是 python 语言编写的供需偏差稳健处理与六维供应商指标构造核心代码

```
def winsorize_supply_gap(data, quantile=0.995):
    result = data.copy()
    active = result["订货款"].gt(0)

    # 供货量相对订货量的有符号偏差
    delta = (
        result.loc[active, "供货量"]
        - result.loc[active, "订货款"]
    )

    # 99.5%分位数作为异常偏差上界
    gap_limit = delta.abs().quantile(quantile)

    # 保留偏差方向进行 Winsor 缩尾
    clipped_delta = np.sign(delta) * np.minimum(
        delta.abs(), gap_limit
    )

    # 重构截尾后的实际供货量
    result["截尾后供货量"] = 0.0
    result.loc[active, "截尾后供货量"] = np.maximum(
        result.loc[active, "订货款"].to_numpy()
        + clipped_delta.to_numpy(),
        0.0
    )

    return result, gap_limit

def build_supplier_indicators(data, horizon=240, satisfy_ratio=0.90):
    records = []

    for sid, block in data.groupby("供应商 ID"):
        material = block["材料分类"].iloc[0]
        order_qty = block["订货款"].to_numpy(dtype=float)
        supply_qty = block["截尾后供货量"].to_numpy(dtype=float)

        active_supply = supply_qty[supply_qty > 0]
        supply_weeks = len(active_supply)
```



```

# ① 总供货能力
total_supply = supply_qty.sum()

# ② 平均供货强度
mean_supply = (
    total_supply / supply_weeks
    if supply_weeks > 0 else 0.0
)

# ③ 供货波动系数
supply_cv = (
    np.std(active_supply, ddof=1) / mean_supply
    if supply_weeks > 1 and mean_supply > 0
    else 0.0
)

ordered = order_qty > 0
total_order = order_qty[ordered].sum()
order_weeks = ordered.sum()

# ④ 到货率
arrival_ratio = (
    total_supply / total_order
    if total_order > 0 else 0.0
)

# ⑤ 订单满足率
satisfaction_ratio = (
    np.sum(
        ordered
        & (supply_qty >= satisfy_ratio * order_qty)
    ) / order_weeks
    if order_weeks > 0 else 0.0
)

# ⑥ 供货覆盖率
coverage_ratio = supply_weeks / horizon

records.append({
    "供应商 ID": sid,
    "材料分类": material,
    "总供货能力": total_supply,
    "平均供货强度": mean_supply,

```

```

        "供货波动系数": supply_cv,
        "到货率": arrival_ratio,
        "订单满足率": satisfaction_ratio,
        "供货覆盖率": coverage_ratio
    })

    return pd.DataFrame(records)

df_clean, winsor_limit = winsorize_supply_gap(
    df, quantile=0.995
)

supplier_indicators = build_supplier_indicators(
    df_clean,
    horizon=240,
    satisfy_ratio=0.90
)

```

附录 2

介绍：该代码是 python 语言编写的 CRITIC-熵权组合赋权核心代码

```

#CRITIC-熵权组合赋权核心代码
# CRITIC-熵权组合赋权
def calculate_combined_weights(indicator_table):
    indicators = [
        "总供货能力",
        "平均供货强度",
        "供货波动系数",
        "到货率",
        "订单满足率",
        "供货覆盖率"
    ]
    negative_index = {"供货波动系数"}
    X = indicator_table[indicators].astype(float)
    # 1. 指标同向化及极差标准化
    Z = pd.DataFrame(index=X.index)
    for name in indicators:
        xmin, xmax = X[name].min(), X[name].max()

        if np.isclose(xmax, xmin):
            Z[name] = 0.0
        elif name in negative_index:
            Z[name] = (xmax - X[name]) / (xmax - xmin)

```

```

        else:
            Z[name] = (X[name] - xmin) / (xmax - xmin)

# 2. CRITIC 客观权重
contrast = Z.std(ddof=1)
correlation = Z.corr()
conflict = (1 - correlation).sum(axis=1)

critic_info = contrast * conflict
weight_critic = critic_info / critic_info.sum()

# 3. 熵权法
proportion = Z.div(
    Z.sum(axis=0) + 1e-12,
    axis=1
)

entropy = pd.Series(index=indicators, dtype=float)
n = len(Z)

for name in indicators:
    p = proportion[name].to_numpy()
    p = p[p > 0]

    entropy[name] = (
        -(p * np.log(p)).sum()
        / np.log(n)
    )

difference = 1 - entropy
weight_entropy = difference / difference.sum()

# 4. 对两种客观权重进行等距离最小二乘融合
weight_combined = (
    weight_critic + weight_entropy
) / 2

weights = pd.DataFrame({
    "指标": indicators,
    "CRITIC 权重": weight_critic.values,
    "熵权法权重": weight_entropy.values,
    "组合权重": weight_combined.values
})

return Z, weights

```

```
normalized_matrix, final_weights = calculate_combined_weights(
    supplier_indicators
)
```

附录 3

介绍：该代码是 python 语言编写的三状态随机需求建模与 Monte Carlo 情景生成核心代码

```
def build_demand_model(order_table, selected_suppliers, scale=1.5):
    sample = order_table[
        order_table["供应商 ID"].isin(selected_suppliers)
    ].copy()

    week_cols = list(sample.columns[2:])
    # ----- 1. 240 周历史需求整理 -----
    history = sample.melt(
        id_vars=["供应商 ID", "材料分类"],
        value_vars=week_cols,
        var_name="历史周次",
        value_name="订货量"
    )
    history["订货量"] = pd.to_numeric(
        history["订货量"],
        errors="coerce"
    ).fillna(0.0)
    week_map = {
        week: i + 1
        for i, week in enumerate(week_cols)
    }
    history["历史周序号"] = history["历史周次"].map(week_map)
    weekly = (
        history
        .groupby(
            ["历史周序号", "材料分类"],
            as_index=False
        )["订货量"]
        .sum()
        .rename(columns={"订货量": "历史需求"})
    )
    # 240 周划分为 5 个 48 周周期
    weekly["历史周期"] = (
        (weekly["历史周序号"] - 1) // 48 + 1
```

```

)
weekly["周次"] = (
    (weekly["历史周序号"] - 1) % 48 + 1
)
# ----- 2. 历史同期需求参数 -----
param = (
    weekly
    .groupby(["周次", "材料分类"])["历史需求"]
    .agg(
        D0_m3="mean",
        sigma_m3=lambda x: x.std(ddof=1)
    )
    .reset_index()
)
param["CV"] = np.where(
    param["D0_m3"] > 0,
    param["sigma_m3"] / param["D0_m3"],
    0.0
)
# ----- 3. H/M/L 三状态识别 -----
state_history = weekly.merge(
    param,
    on=["周次", "材料分类"],
    how="left"
)
D0 = state_history["D0_m3"]
sigma = state_history["sigma_m3"]
q = state_history["历史需求"]

state_history["需求状态"] = np.select(
    [
        q >= D0 + scale * sigma,
        q <= D0 - scale * sigma
    ],
    ["H", "L"],
    default="M"
)
# ----- 4. H/M/L 需求水平 -----
param["D_H_m3"] = (
    param["D0_m3"]
    + scale * param["sigma_m3"]
)
param["D_M_m3"] = param["D0_m3"]
param["D_L_m3"] = np.maximum(

```

```

        param["D0_m3"]
        - scale * param["sigma_m3"],
        0.0
    )
    # 未来仅规划 24 周
    future_param = param[
        param["周次"] <= 24
    ].copy()
    return future_param
def generate_demand_paths(
    scenario_table,
    path_num=500,
    weeks=24,
    seed=2026
):
    states = np.array(["H", "M", "L"])
    # 历史样本估计得到的最终统一状态概率
    probabilities = np.array([
        0.10,
        0.88,
        0.02
    ])
    rng = np.random.default_rng(seed)
    # 每条路径包含未来 24 周的宏观需求状态
    state_paths = rng.choice(
        states,
        size=(path_num, weeks),
        p=probabilities
    )
    demand_map = {}
    for _, row in scenario_table.iterrows():
        demand_map[
            (int(row["周次"]), row["材料分类"])
        ] = {
            "H": row["D_H_m3"],
            "M": row["D_M_m3"],
            "L": row["D_L_m3"]
        }
    records = []
    for r in range(path_num):
        for t in range(1, weeks + 1):
            # 同一周 A/B/C 共享同一个宏观需求状态
            state = state_paths[r, t - 1]
            for material in ["A", "B", "C"]:

```

```

        records.append({
            "情景编号": r + 1,
            "周次": t,
            "材料分类": material,
            "需求状态": state,
            "需求量": demand_map[
                (t, material)
            ][state]
        })
    return state_paths, pd.DataFrame(records)

future_param = build_demand_model(
    order_df,
    set(top50["供应商 ID"]),
    scale=1.5
)
sampled_states, demand_paths = generate_demand_paths(
    future_param,
    path_num=500,
    weeks=24,
    seed=2026
)

```

附录 4

介绍：该代码是 python 语言编写的基于情景的 Mean-CVaR 随机订购优化代码

```

model = pyo.ConcreteModel()
model.I = pyo.Set(initialize=selected_suppliers)
model.T = pyo.RangeSet(1, 24)
model.M = pyo.Set(initialize=["A", "B", "C"])
model.R = pyo.Set(initialize=scenarios)
# 第一阶段共享订购决策
model.x = pyo.Var(model.I, model.T, domain=pyo.NonNegativeReals)
model.y = pyo.Var(model.I, model.T, domain=pyo.Binary)
# 基准库存与安全缺口
model.BInv = pyo.Var(model.T, model.M, domain=pyo.NonNegativeReals)
model.Gap = pyo.Var(model.T, model.M, domain=pyo.NonNegativeReals)
# 情景实现后的库存与缺货
model.Inv = pyo.Var(
    model.R, model.T, model.M,
    domain=pyo.NonNegativeReals
)

```

```

model.Short = pyo.Var(
    model.R, model.T, model.M,
    domain=pyo.NonNegativeReals
)
# CVaR 辅助变量
model.Cost = pyo.Var(model.R, domain=pyo.NonNegativeReals)
model.eta = pyo.Var(domain=pyo.NonNegativeReals)
model.z = pyo.Var(model.R, domain=pyo.NonNegativeReals)
# 供应商订购上下界
model.OrderUpper = pyo.Constraint(
    model.I, model.T,
    rule=lambda m, i, t:
        m.x[i, t] <= Smax[i] * m.y[i, t]
)
model.OrderLower = pyo.Constraint(
    model.I, model.T,
    rule=lambda m, i, t:
        m.x[i, t] >= Qmin[i] * m.y[i, t]
)
# 总周转运力
model.TransportCap = pyo.Constraint(
    model.T,
    rule=lambda m, t:
        sum(m.x[i, t] for i in m.I) <= 48000
)
# 折损后的到货量
model.Arrival = pyo.Expression(
    model.T, model.M,
    rule=lambda m, t, mat:
        (1 - overall_loss) * sum(
            m.x[i, t]
            for i in suppliers_by_material[mat]
        )
)
# 基准库存平衡
def base_balance(m, t, mat):
    previous = 0 if t == 1 else m.BInv[t-1, mat]

    return (
        previous + m.Arrival[t, mat]
        == Dbar[t, mat] + m.BInv[t, mat]
    )
model.BaseBalance = pyo.Constraint(
    model.T, model.M,

```



```

        rule=base_balance
    )
    # 两周安全库存
    model.Safety = pyo.Constraint(
        model.T, model.M,
        rule=lambda m, t, mat:
            m.BInv[t, mat] + m.Gap[t, mat]
            >= 2 * Dbar[t, mat]
    )
    model.GapLimit = pyo.Constraint(
        expr=sum(
            model.Gap[t, mat]
            for t in model.T
            for mat in model.M
        ) <= gap_star + 1e-3
    )
    # 情景库存-缺货平衡
    def scenario_balance(m, r, t, mat):
        previous = 0 if t == 1 else m.Inv[r, t-1, mat]

        return (
            previous
            + m.Arrival[t, mat]
            + m.Short[r, t, mat]
            ==
            D[r, t, mat]
            + m.Inv[r, t, mat]
        )
    model.ScenarioBalance = pyo.Constraint(
        model.R, model.T, model.M,
        rule=scenario_balance
    )
    # 采购成本
    model.PurchaseCost = pyo.Expression(
        expr=sum(
            P[material_of[i]]
            * lambda_i[i]
            * model.x[i, t]
            for i in model.I
            for t in model.T
        )
    )
    # 各情景总成本
    def scenario_cost(m, r):

```

```

    holding = sum(
        h[mat] * m.Inv[r, t, mat]
        for t in m.T for mat in m.M
    )
    shortage = sum(
        g[mat] * m.Short[r, t, mat]
        for t in m.T for mat in m.M
    )
    return (
        m.Cost[r]
        == m.PurchaseCost + holding + shortage
    )
model.ScenarioCost = pyo.Constraint(
    model.R,
    rule=scenario_cost
)
# 95% CVaR 线性化
model.Excess = pyo.Constraint(
    model.R,
    rule=lambda m, r:
        m.z[r] >= m.Cost[r] - m.eta
)
R = len(scenarios)
alpha = 0.95
rho = 0.5
model.ExpectedCost = pyo.Expression(
    expr=sum(model.Cost[r] for r in model.R) / R
)
model.CVaR = pyo.Expression(
    expr=model.eta
    + sum(model.z[r] for r in model.R)
    / ((1 - alpha) * R)
)
model.Obj = pyo.Objective(
    expr=(
        (1 - rho) * model.ExpectedCost
        + rho * model.CVaR
    ),
    sense=pyo.minimize
)

```

介绍：该代码是 python 语言编写的采购结构调整的四阶段词典序线性规划核心代码

```
# 问题三：采购结构调整线性规划模型
def build_order_model(
    suppliers, material_of, Smax,
    suppliers_by_material
):
    model = pyo.ConcreteModel()
    model.I = pyo.Set(initialize=suppliers)
    model.T = pyo.RangeSet(1, 24)
    model.M = pyo.Set(initialize=["A", "B", "C"])
    production = 28200.0
    loss = 0.013881724995
    transport_cap = 48000.0
    d = {
        "A": 0.60,
        "B": 0.66,
        "C": 0.72
    }
    # O_it: 向供应商 i 的订购量
    model.O = pyo.Var(
        model.I, model.T,
        domain=pyo.NonNegativeReals
    )
    # Z_tm: 第 t 周由材料 m 承担的产品产量
    model.Z = pyo.Var(
        model.T, model.M,
        domain=pyo.NonNegativeReals
    )
    # Inv_tm: 材料 m 的期末库存
    model.Inv = pyo.Var(
        model.T, model.M,
        domain=pyo.NonNegativeReals
    )
    # V_t: 两周安全库存等价产品缺口
    model.V = pyo.Var(
        model.T,
        domain=pyo.NonNegativeReals
    )
    # 每周保持满产
    model.Production = pyo.Constraint(
        model.T,
        rule=lambda m, t:
            sum(m.Z[t, k] for k in m.M) == production
```

```

)
# 单家供应商周供货能力
model.SupplierCapacity = pyo.Constraint(
    model.I, model.T,
    rule=lambda m, i, t:
        m.O[i, t] <= Smax[i]
)
# 8家转运商合计周运力
model.TransportCapacity = pyo.Constraint(
    model.T,
    rule=lambda m, t:
        sum(m.O[i, t] for i in m.I) <= transport_cap
)
# 原材料库存动态平衡
def inventory_rule(m, t, material):
    received = (1 - loss) * sum(
        m.O[i, t]
        for i in suppliers_by_material[material]
    )
    consumed = d[material] * m.Z[t, material]
    previous = (
        0 if t == 1
        else m.Inv[t - 1, material]
    )
    return (
        m.Inv[t, material]
        == previous + received - consumed
    )
model.InventoryBalance = pyo.Constraint(
    model.T, model.M,
    rule=inventory_rule
)
# 三类库存统一折算为可支持的产品产量
def safety_rule(m, t):
    equivalent_inventory = sum(
        m.Inv[t, k] / d[k]
        for k in m.M
    )
    return (
        equivalent_inventory + m.V[t]
        >= 2 * production
    )
model.SafetyStock = pyo.Constraint(
    model.T,

```

```

        rule=safety_rule
    )
    return model
model = build_order_model(
    SUPPLIERS,
    supplier_material,
    Smax,
    I_m
)
# 第一层：优先保证两周安全库存
model.Obj1 = pyo.Objective(
    expr=sum(model.V[t] for t in model.T),
    sense=pyo.minimize
)
solver.solve(model)

V_star = pyo.value(model.Obj1)

model.LockV = pyo.Constraint(
    expr=sum(model.V[t] for t in model.T)
    <= V_star + 1e-3
)
model.Obj1.deactivate()
# 第二层：在库存保障不恶化时，多采购 A、少采购 C
model.Obj2 = pyo.Objective(
    expr=sum(
        model.Z[t, "C"] - model.Z[t, "A"]
        for t in model.T
    ),
    sense=pyo.minimize
)
solver.solve(model)
AC_star = pyo.value(model.Obj2)

model.LockAC = pyo.Constraint(
    expr=sum(
        model.Z[t, "C"] - model.Z[t, "A"]
        for t in model.T
    )
    <= AC_star + 1e-3
)
model.Obj2.deactivate()
# 第三层：减少 24 周总订购规模
model.Obj3 = pyo.Objective(

```

```

    expr=sum(
        model.O[i, t]
        for i in model.I
        for t in model.T
    ),
    sense=pyo.minimize
)
solver.solve(model)
Q_star = pyo.value(model.Obj3)
model.LockQ = pyo.Constraint(
    expr=sum(
        model.O[i, t]
        for i in model.I
        for t in model.T
    )
    <= Q_star + 1e-3
)
model.Obj3.deactivate()
# 第四层：在前三层最优基础上压缩库存占用
model.Obj4 = pyo.Objective(
    expr=sum(
        model.Inv[t, m]
        for t in model.T
        for m in model.M
    ),
    sense=pyo.minimize
)
solver.solve(model

```

附录 6

介绍：该代码是 python 语言编写的自适应损耗率预测与动态两阶段转运优化核心代码

```

# ----- 1. 样本自适应收缩预测 -----
def predict_loss(values, week, train_years, kappa):
    # 历年同期样本
    idx = [
        week - 1 + 48 * y
        for y in range(train_years)
    ]

```

```

same = np.asarray(values[idx], dtype=float)
weights = np.arange(1, train_years + 1, dtype=float)
valid = np.isfinite(same) & (same > 0)
same = same[valid]
weights = weights[valid]
# 长期稳健基准: 训练期非零损耗率中位数
history = np.asarray(
    values[:48 * train_years],
    dtype=float
)
history = history[
    np.isfinite(history) & (history > 0)
]
baseline = (
    np.median(history)
    if len(history) > 0 else np.nan
)
if len(same) == 0:
    return baseline
# 近期年份赋予更高权重
seasonal = np.average(
    same,
    weights=weights
)
# 样本量自适应收缩
alpha = len(same) / (len(same) + kappa)
if not np.isfinite(baseline):
    return seasonal
return (
    alpha * seasonal
    + (1 - alpha) * baseline
)
# ----- 2. 验证集选择收缩参数 -----
kappa_grid = [
    0, 0.25, 0.5, 0.75, 1,
    1.5, 2, 3, 4, 5, 7, 10, 15, 20
]

records = []
for kappa in kappa_grid:
    validation = pd.concat([
        evaluate_year(3, kappa),
        evaluate_year(4, kappa)
    ], ignore_index=True)

```

```

valid = validation[
    validation["是否有效测试"]
]
records.append({
    "kappa": kappa,
    "MAPE": valid["APE"].mean(),
    "MAE": valid["绝对误差"].mean()
})
score = pd.DataFrame(records)

for col in ["MAPE", "MAE"]:
    low = score[col].min()
    high = score[col].max()
    score[col + "_norm"] = (
        (score[col] - low) / (high - low)
    )
score["综合误差"] = (
    0.5 * score["MAPE_norm"]
    + 0.5 * score["MAE_norm"]
)
best_kappa = score.loc[
    score["综合误差"].idxmin(),
    "kappa"
]

# ----- 3. 预测未来 24 周各转运商损耗率 -----
loss_hat = {}

for _, row in loss_df.iterrows():
    carrier = row["转运商 ID"]
    history = row[week_cols].to_numpy(dtype=float)

    for t in range(1, 25):
        loss_hat[carrier, t] = (
            predict_loss(
                history,
                week=t,
                train_years=5,
                kappa=best_kappa
            ) / 100.0
        )

# ----- 4. 动态损耗率下逐周两阶段转运 MILP -----
def solve_week_transport(t, order_t, loss_hat, carriers, cap=6000):
    suppliers = list(order_t.keys())

```



```

model = pyo.ConcreteModel()
model.I = pyo.Set(initialize=suppliers)
model.K = pyo.Set(initialize=carriers)
model.S = pyo.Var(
    model.I, model.K,
    domain=pyo.NonNegativeReals
)
model.w = pyo.Var(
    model.I, model.K,
    domain=pyo.Binary
)
# 各供应商订单全部完成运输
model.FullTransport = pyo.Constraint(
    model.I,
    rule=lambda m, i:
        sum(m.S[i, k] for k in m.K)
        == order_t[i]
)
# 单家转运商周运力
model.Capacity = pyo.Constraint(
    model.K,
    rule=lambda m, k:
        sum(m.S[i, k] for i in m.I)
        <= cap
)
# 运输量与连接变量联动
model.Link = pyo.Constraint(
    model.I, model.K,
    rule=lambda m, i, k:
        m.S[i, k]
        <= min(cap, order_t[i]) * m.w[i, k]
)
# 动态预测损耗
model.WeeklyLoss = pyo.Expression(
    expr=sum(
        model.S[i, k] * loss_hat[k, t]
        for i in model.I
        for k in model.K
    )
)
# 第一阶段：最低运输损耗
model.ObjLoss = pyo.Objective(
    expr=model.WeeklyLoss,
    sense=pyo.minimize
)

```

```

)
solver.solve(model)
loss_star = pyo.value(model.WeeklyLoss)
# 第二阶段: 锁定最低损耗, 减少运输连接
model.ObjLoss.deactivate()
model.LockLoss = pyo.Constraint(
    expr=model.WeeklyLoss
    <= loss_star + 1e-4
)
model.ObjLinks = pyo.Objective(
    expr=sum(
        model.w[i, k]
        for i in model.I
        for k in model.K
    ),
    sense=pyo.minimize
)
solver.solve(model)
return model

# ----- 5. 逐周求解未来 24 周转运方案 -----
transport_models = {}

for t in range(1, 25):
    order_t = {
        row["供应商 ID"]: row["订购量"]
        for _, row in order_long[
            order_long["周次"] == t
        ].iterrows()
    }
    if order_t:
        transport_models[t] = solve_week_transport(
            t,
            order_t,
            loss_hat,
            CARRIERS
        )

```

附录 7

介绍: 该代码是 python 语言编写的联合产能边界与多目标产能扫描决策核心代码

```

materials = ["A", "B", "C"]
d = {"A": 0.60, "B": 0.66, "C": 0.72}
transport_cap = 8 * 6000

```

```

# ----- 1. 供应-转运联合理论产能边界 -----
def solve_capacity_boundary(material_caps):
    # 最大化折损后原材料可支撑的产品产量
    c = np.array([
        -(1 - avg_loss) / d[m]
        for m in materials
    ])
    result = linprog(
        c=c,
        A_ub=[[1, 1, 1]],
        b_ub=[transport_cap],
        bounds=[
            (0, material_caps[m])
            for m in materials
        ],
        method="highs"
    )
    if not result.success:
        raise RuntimeError(result.message)
    return -result.fun
# 各类材料历史极值形成跨周组合理论上界
theory_cap = solve_capacity_boundary(
    material_max_cap
)
scan_upper = (
    np.floor((theory_cap / 10000) / 0.2)
    * 0.2
)
capacity_candidates = np.concatenate([
    [2.82],
    np.arange(3.0, scan_upper + 1e-9, 0.2)
]) * 10000
# ----- 2. 给定候选产能的订购-生产-库存模型 -----
def solve_capacity_plan(CT):
    model = pyo.ConcreteModel()
    model.T = pyo.Set(initialize=weeks)
    model.M = pyo.Set(initialize=materials)
    model.Q = pyo.Var(
        model.T, model.M,
        domain=pyo.NonNegativeReals
    )
    model.Z = pyo.Var(
        model.T, model.M,
        domain=pyo.NonNegativeReals
    )

```

```

)
model.Inv = pyo.Var(
    model.T, model.M,
    domain=pyo.NonNegativeReals
)
model.V = pyo.Var(
    model.T,
    domain=pyo.NonNegativeReals
)
# 每周完成目标产能
model.Production = pyo.Constraint(
    model.T,
    rule=lambda m, t:
        sum(m.Z[t, k] for k in materials) == CT
)
# 材料供应能力
model.MaterialCap = pyo.Constraint(
    model.T, model.M,
    rule=lambda m, t, k:
        m.Q[t, k] <= material_cap[k]
)
# 总周转运力
model.TransportCap = pyo.Constraint(
    model.T,
    rule=lambda m, t:
        sum(m.Q[t, k] for k in materials)
        <= transport_cap
)
# 库存动态平衡
def inventory_rule(m, t, k):
    arrival = (
        (1 - avg_loss)
        * m.Q[t, k]
    )
    consume = d[k] * m.Z[t, k]
    previous = (
        0 if t == 1
        else m.Inv[t-1, k]
    )
    return (
        m.Inv[t, k]
        == previous + arrival - consume
    )
model.Inventory = pyo.Constraint(

```

```

        model.T, model.M,
        rule=inventory_rule
    )
    # 两周安全库存
    model.Safety = pyo.Constraint(
        model.T,
        rule=lambda m, t:
            sum(
                m.Inv[t, k] / d[k]
                for k in materials
            ) + m.V[t] >= 2 * CT
    )
    # ----- 词典序目标 1: 安全缺口最小 -----
    gap = sum(
        model.V[t]
        for t in model.T
    )
    model.Obj1 = pyo.Objective(
        expr=gap,
        sense=pyo.minimize
    )
    solver.solve(model)
    gap_star = pyo.value(gap)

    model.LockGap = pyo.Constraint(
        expr=gap <= gap_star + 1e-4
    )
    model.Obj1.deactivate()
    # ----- 目标 2: 少用 C -----
    C_amount = sum(
        model.Z[t, "C"]
        for t in model.T
    )
    model.Obj2 = pyo.Objective(
        expr=C_amount,
        sense=pyo.minimize
    )
    solver.solve(model)
    C_star = pyo.value(C_amount)
    model.LockC = pyo.Constraint(
        expr=C_amount <= C_star + 1e-4
    )
    model.Obj2.deactivate()
    # ----- 目标 3: 多用 A -----

```

```

A_amount = sum(
    model.Z[t, "A"]
    for t in model.T
)
model.Obj3 = pyo.Objective(
    expr=-A_amount,
    sense=pyo.minimize
)
solver.solve(model)
A_star = pyo.value(A_amount)
model.LockA = pyo.Constraint(
    expr=A_amount >= A_star - 1e-4
)
model.Obj3.deactivate()
# ----- 目标 4: 采购成本最低 -----
purchase_cost = sum(
    price[k] * model.Q[t, k]
    for t in model.T
    for k in model.M
)
model.Obj4 = pyo.Objective(
    expr=purchase_cost,
    sense=pyo.minimize
)
solver.solve(model)
cost_star = pyo.value(purchase_cost)
model.LockCost = pyo.Constraint(
    expr=purchase_cost
    <= cost_star + 1e-4
)
model.Obj4.deactivate()
# ----- 目标 5: 库存占用最低 -----
model.Obj5 = pyo.Objective(
    expr=sum(
        price[k] * model.Inv[t, k]
        for t in model.T
        for k in model.M
    ),
    sense=pyo.minimize
)
solver.solve(model)
return model
# ----- 3. 候选产能逐档扫描 -----
results = []

```

```

for CT in capacity_candidates:
    model = solve_capacity_plan(CT)
    aggregate_plan = extract_plan(model)
    order_plan = allocate_to_suppliers(
        CT,
        aggregate_plan
    )
    transport_plan = solve_transport_fast(
        CT,
        order_plan
    )
    metrics = evaluate_candidate(
        CT,
        order_plan,
        transport_plan
    )
    results.append(metrics)
summary = pd.DataFrame(results)
# ----- 4. 成本、风险及安全边界 -----
summary["边际成本弹性"] = (
    summary["总成本"].pct_change()
    /
    summary["候选产能_m3"].pct_change()
)
summary["HHI 变化率"] = (
    summary["HHI"].pct_change()
)
# 最大连续稳态安全产能
safe_capacity = []
for _, row in summary.iterrows():
    if row["稳态安全库存是否满足"]:
        safe_capacity.append(
            row["候选产能_m3"]
        )
    else:
        break
CT_safe = max(safe_capacity)
# 安全区间内成本拐点
CT_cost = np.nan
for i in range(1, len(summary) - 1):
    window = summary.iloc[i:i+2]
    if (
        (window["候选产能_m3"] <= CT_safe).all()
        and

```

```

        (window["边际成本弹性"] > 1.05).all()
    ):
        CT_cost = window.iloc[0]["候选产能_m3"]
        break
# 安全区间内 HHI 风险拐点
risk_rows = summary[
    (summary["候选产能_m3"] <= CT_safe)
    &
    (summary["HHI 变化率"] > 0.15)
]
CT_risk = (
    risk_rows.iloc[0]["候选产能_m3"]
    if len(risk_rows) > 0
    else np.nan
)
# ----- 5. 最终推荐产能 -----
CT_cost_eff = (
    CT_cost
    if np.isfinite(CT_cost)
    else CT_safe
)
CT_risk_eff = (
    CT_risk
    if np.isfinite(CT_risk)
    else CT_safe
)
CT_recommend = min(
    CT_safe,
    CT_cost_eff,
    CT_risk_eff
)

```